

前 言

MATLAB 的首创者 Cleve Moler 博士在数值分析,特别是在数值线性代数的领域中很有影响。他参与编写了数值分析领域一些著名的著作。1980 年前后, Moler 博士在 New Mexico 大学讲授线性代数课程时,发现用其它高级语言编程极为不便,便构思并开发了 MATLAB (MATrix LABoratory, 即矩阵实验室),这一软件利用了当时流行的 EISPACK(基于特征值计算的软件包)和 LINPACK(线性代数软件包)两大软件包中可靠的子程序,用 FORTRAN 语言编写了集命令翻译、科学计算于一身的一套交互式软件系统。现在的 MATLAB 已经用 C 语言作了完全的改写。在 MATLAB 下矩阵的运算变得异常的容易,后来的版本中又增添了丰富多彩的图形图像处理及多媒体功能。这一系统逐渐发展、完善,逐步走向成效,形成了今天的模样。

严格地说, MATLAB 并不是一种计算机语言,因为用它编写出来的程序并不能脱离 MATLAB 环境而执行,但从其功能上来说, MATLAB 已经完全具备了计算机语言的结构与性能,所以我们称 MATLAB 为语言。MATLAB 和其它级语言之间的关系仿佛高级语言和汇编语言的关系一样,因为高级语言的执行效率要低于汇编语言,然而其编程效率与可读性、可移植性要远远高于汇编语言。同样 MATLAB 比一般高级语言的执行效率要低,而其编程效率与可读性、可移植性要远远高于其它高级语言,所以在计算机辅助设计与仿真中较适合于从像 MATLAB 这样的专用高级语言入手,这不但可以大大地提高编程的效率,而且可以大大地提高编程的质量与可靠性。

值得指出的是, MATLAB 一开始并不是为控制理论与系统的设计者们编写的。但 MATLAB 软件一出现很快就引起了控制界研究人员的瞩目,因为它把看起来那么繁琐的矩阵操作变得简单得令人难以置信,同时 MATLAB 还可以十分容易地绘制各种精美的图形。此外, MATLAB 又有那么好的可扩充性,可以把它当作一种更高级的语言那样去使用,用户可以用它容易地编写出各种通用或专用应用程序来。

由于 MATLAB 提供了强大的矩阵处理和绘图功能,很多控制界的名家在自己擅长的领域编写了一些具有特殊意义的 MATLAB 工具箱,如美国学者 Alanlaub 与 John Little 编写的控制系统工具箱(control system toolbox),美国学者 John little 和 Loren Shure 编写的信号处理工具箱(signal processing toolbox),瑞典学者 leonard Ljung 的系统辨识工具箱(system identification toolbox),美国籍学者 Bichard Chiang 与 Michael Sofanov 的鲁棒控制工具箱(robust control toolbox), μ 分析与综合工具箱(μ -analysis and synthesis toolbox),美国学者 Craig Borghesani, Yossi Chait 和 Oded Yaniv 设计的定量反馈理论工具箱(QFT toolbox)、美国学者 Howard Demnith 与 Mark Beale 的神经网络工具箱(neural network toolbox),英国学者 Jan Maciejowski 等的多变量领域设计工具箱(multivariable frequency design toolbox),英国学者 Peter Fleming 及美国学者 Andrew Crace 写的两个版本的最优化工具箱(optimization toolbox)等等,1992 年 Math Works 公司推出的交互式模型输入与仿真环境 SIMULINK 更使得 MATLAB 为控制系统的仿真与 CAD 中的应用打开了崭新的局面。目前, MATLAB 已经成为国际上最流行的控制系统计算机辅助设计的软件工具,现有的 MATLAB 已经不仅仅是一个“矩阵实验室”了,它已经成为了一种具有广泛应用前景的全新的计算机高级编程语言了。

Math Works 公司于 1984 年推出了第一个 MATLAB 的商业版本。当时的 MATLAB 版本已经用 C 语言作了完全的改写,其后又增添了丰富多彩的图形图像处理、多媒体功能、符号运算和它与其他流行软件的接口功能,使得 MATLAB 的功能越来越强大。最早的 PC 机版又称为 PC-MATLAB,其工作站版本又称为 Pro MATLAB。1990 年推出的 MATLAB 35i

版是第一个可以运行于 Microsoft windows 下的版本，它可以在两个窗口上分别显示命令行计算结果和图形结果。稍后推出的 SimuLAB 环境首次引入了基于框图的仿真功能，其模型输入的方式令人耳目一新，该环境就是我们现在所知的 Simulink。

The Math Works 公司于 1992 年推出了具有划时代意义的 MATLAB 4.0 版本，并于 1993 年推出了其微机版，充分支持在 Microsoft Windows 进行界面编程。1994 年推出的 4.2 版本扩充了 4.0 版本的功能，尤其在图形界面设计方面更提供了新的方法。

1997 年推出的 MATLAB5.0 版支持了更多的数据结构，如单元数据、数据结构体、多维数组、对象与类等，使其成为一种更方便、完美的编程语言。1999 年初推出的 MATLAB 5.3 版在很多方面又进一步改进了 MATLAB 语言的功能，随之推出的全新版本的最优化工具箱和 Simulink3.0 版达到了很高的档次。2000 年 10 月，MATLAB 6.0 问世，在操作界面上有了很大改观，同时还给出了程序发布窗口，历史信息窗口和变量管理窗口等，为用户的使用提供了很大的方便；在计算内核上抛弃了其一直使用的 LINPACK 和 EISPACK，而采用了更具优势的 LAPACK 软件包和 FFTW 系统，速度变得更快，数值性能也更好；在用户图形界面设计上也更趋合理；与 C 语言接口及转换的兼容性也更强；与之配套的 Simulink4.0 版的新功能也特别引人注目。2001 年 6 月推出的 MATLAB 6.1 版及 Simulink 4.1 版中新的虚拟现实工具箱更给仿真结果三维视景下显示带来了新的解决方案。2002 年 6 月正式推出的 MATLAB Release 13，即 MATLAB 6.5/Simulink 5.0 功能更加强大。

目前，MATLAB 已经成为国际上最流行的科学与工程计算的软件工具，现在的 MATLAB 已经不仅仅是一个“矩阵实验室”了，它已经成为了一种具有广泛应用前景的、全新的计算机高级编程语言了，有人称它为“第四代”计算机语言，它在国内外高校和研究部门正扮演着重要的角色。MATLAB 语言的功能也越来越强大，不断适应新的要求提出新的解决方法。另外，很多长期以来对 MATLAB 有一定竞争能力的软件(如 Matrix-X)已经被 The Math Works 公司吞并，所以可以预见，在科学运算与系统仿真领域 MATLAB 语言将长期保持其独一无二的地位。

下面我们就将介绍一下有关 MATLAB 的安装，工作窗口以及菜单操作，M 文件的编辑调试环境等。

一. MATLAB 的安装

运行 Matlab 安装程序，将进入如图 2.1 Matlab 安装界面，点击“Next”按钮，进入如图 2.2 输入个人版权密码界面。在文本框内输入 PLP 码后，点击“Next”按钮进入到如图 2.3



图 2.1 Matlab 安装界面



图 2.2 输入 Personal Liscence Password 对话框

版权阅读界面，点击“ Yes ”同意后，出现如图 2.4 用户信息对话框。将用户信息输入完毕后，点击“Next”，将会出现产品清单对话框，如图 2.5 所示。在这里您可以设置安装路径、选择安装选择安装选项、修改安装语言版本、选择安装部件以及帮助文档。点击“Next”后，Matlab 安装程序就会进入安装进度显示界面（如图 2.6），按照您的要求进行安装了。

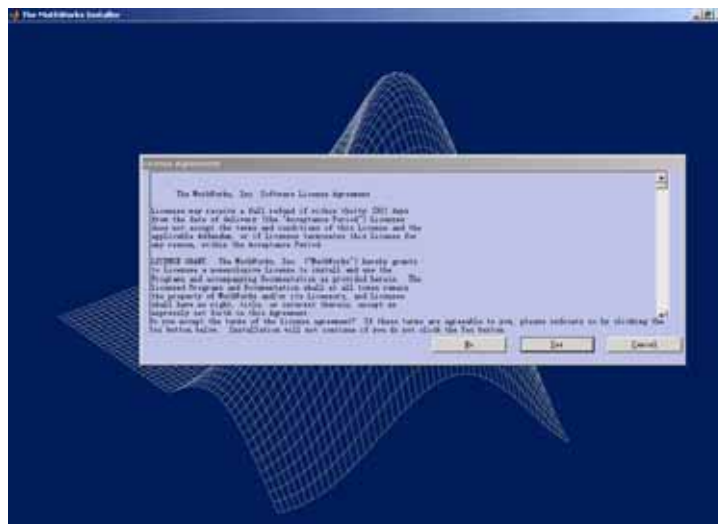


图 2.3 Matlab 版权阅读页



图 2.4 用户信息输入对话框



图 2.5 产品清单列表

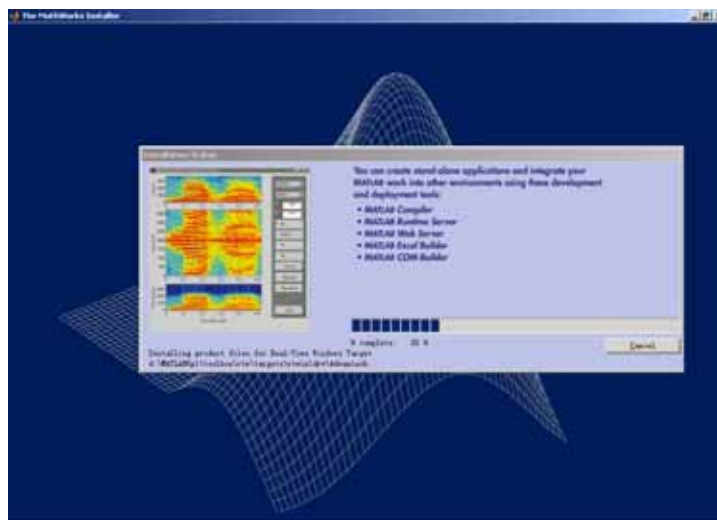


图 2.6 安装进度显示界面

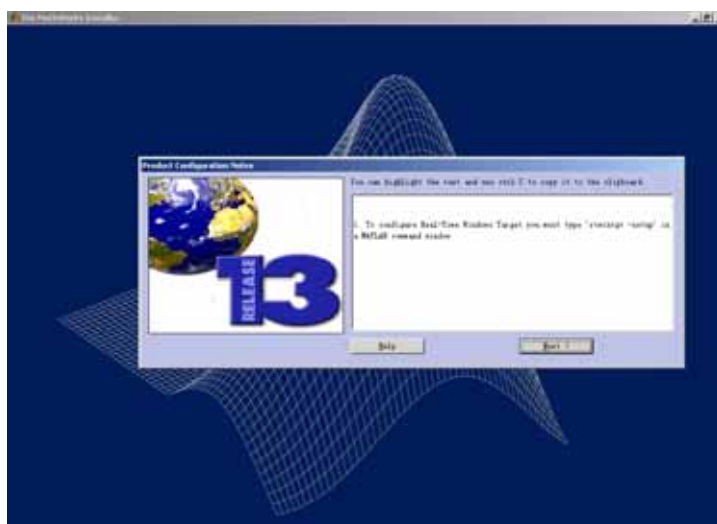


图 2.7 配置实时窗口目标

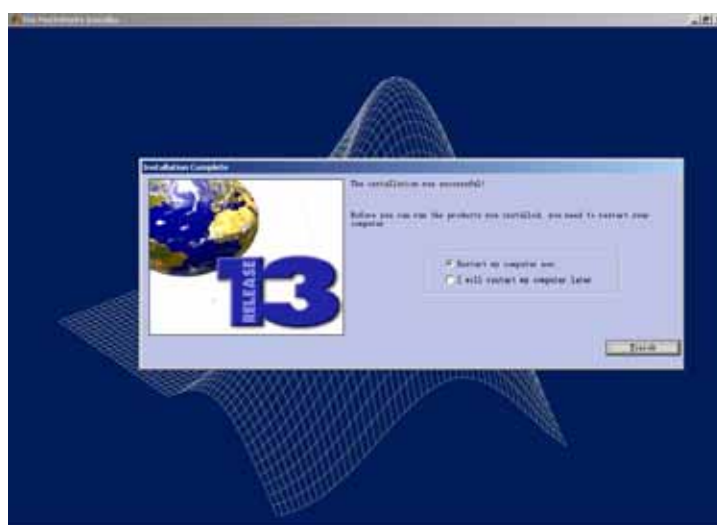


图 2.8 重新启动

在安装完成后，安装程序提示你配置实时窗口目标，如图 2.7 所示。点击“Next”后安装程序提示你重新启动计算机，如图 2.8 所示。点击“Finish”系统将重新启动，至此 Matlab

软件已经成功的被安装到您的计算机中了。

二. MATLAB 的工作窗口和菜单命令

2.1 菜单命令

Matlab 是标准的 Windows 界面，可以利用菜单中的命令完成对工作窗口的操作。它的使用方法与 Windows 的一般应用程序相同，参看图 2.1。

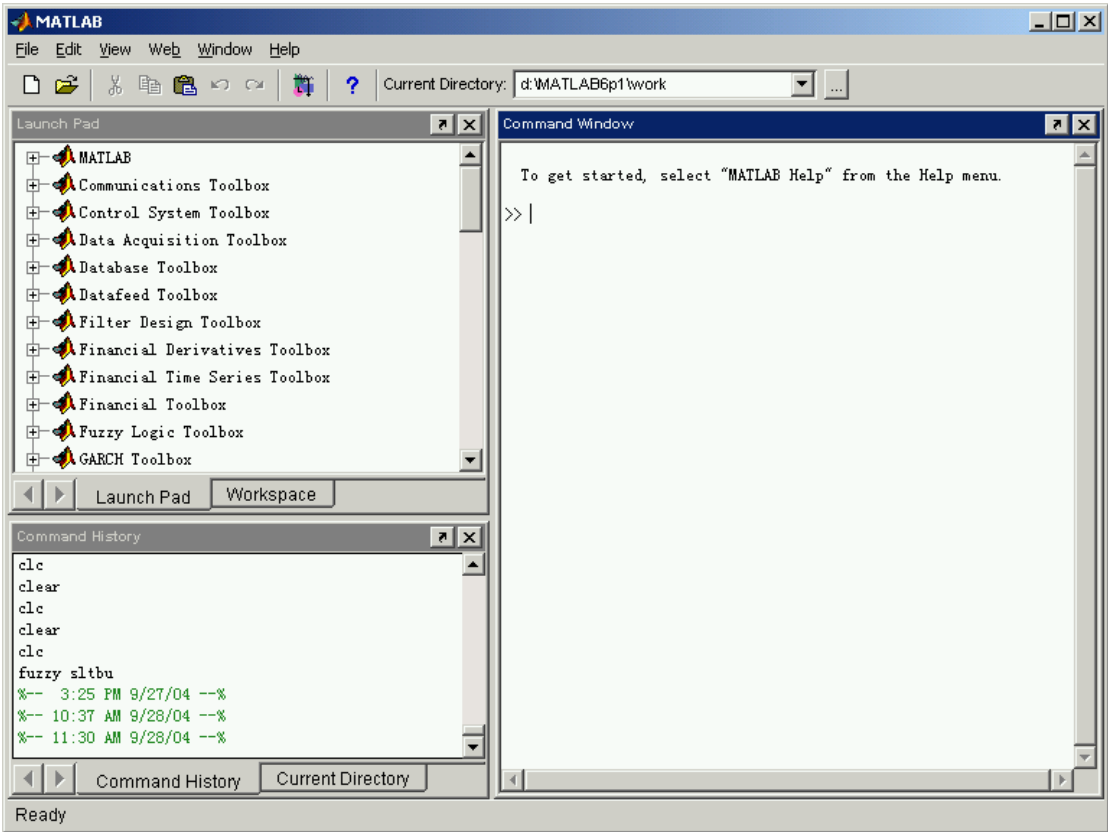


图 2.1 MATLAB 的工作窗口

下面首先对菜单进行介绍。考虑到用户的实际情况，这里只介绍到二级菜单，某些复杂操作将在后面的内容里面介绍。

✧ **File（文件）菜单的内容**

File 菜单的命令如图 2.2 所示，其意义如表 2.1 所示。在表 2.1 中列出了 File 菜单的全部命令行的意义，下面我们来单独介绍一下 Preferences 命令行。

表 2.1 File（文件）菜单的内容

菜单命令	功 能
New	新建一个 M 文件,图形或 Simulink 模块或 GUI。
Open Ctrl+O	打开一个已有的文件(M 文件、图形或 Simulink 模块)。
Close Command Windows Ctrl +W	关闭 Command Windows 子窗口。
Import	导入外部文件数据。
Save Workspace As Ctrl +S	将当前工作空间另存为其他文件。
Set Path	设置 Matlab 工作、搜索路径。

Preferences	设置 Matlab 工作环境参数。
Print	打印输出。
Print Selection	打印所选中的文本或其他对象。
<u>1</u> D:\...6.1\work\ctrltbu2.m	已经打开过的文档，一般保存 4 个文档的路径。
Exit MATLAB Ctrl+Q	退出 Matlab 系统。

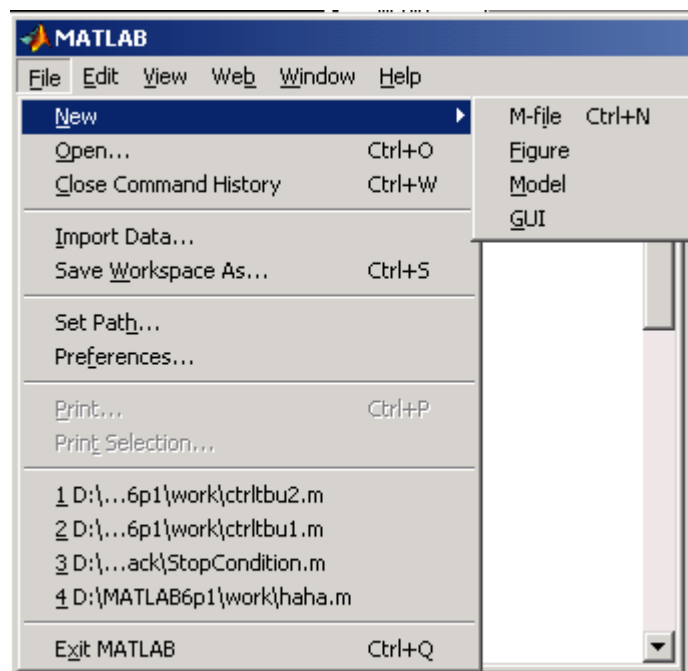


图 2.2 File（文件）菜单命令

在 MATLAB 主窗口选择 File→Preferences 命令，可以在打开的对话框中设置 MATLAB 的工作环境参数。选择 Command Windows 选项，则可以设置 Command Windows 窗口的参数，如图 2.3 所示。下面我们来介绍一下各个选项的含义。

- Text display 选项组

该选项组用来设置 Command Windows 窗口的文本显示格式。

 - ✓ Numeric Format 下拉列表框用来设置 Command Windows 窗口的数值显示格式，其各个选项含义如下：
 - short 5 位定点表示
 - short E 5 位浮点表示
 - short G 系统选择 short 和 short E 中最好的表示
 - long 15 位定点表示
 - long E 15 位浮点表示
 - long G 系统选择 long 和 long E 中最好的表示
 - bank 用元、角、分（美制）定点表示
 - + 仅显示数值的符号
 - ration 近似的有理数表示
 - hex 十六进制表示
 - ✓ Numeric display 下拉列表框用来控制 Command Windows 窗口的数据输出风格，其各个选项含义如下：
 - loose 在显示结果中加入一些空行

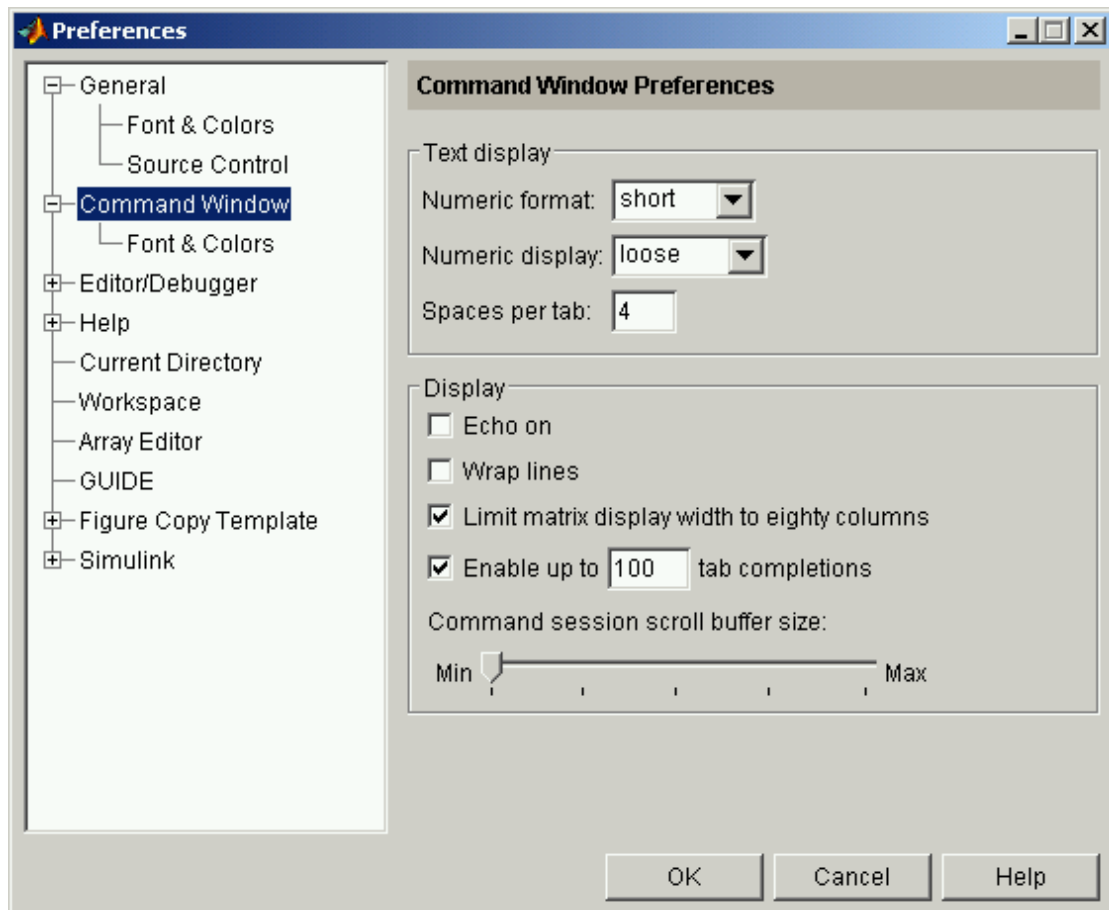


图 2.3 Command Windows 窗口的 Preferences 对话框

- compact 压缩掉显示结果中加入的空行
- ✓ Spaces per tab 编辑框用来设置 tab 键跳转的空格数目，缺省为 4。
- Display 选项组
 - ✓ Echo on 复选框用来选择运行 MATLAB 程序时是否在 Command Windows 窗口显示正在运行的命令；
 - ✓ Wrap lines 复选框用来选择运行 MATLAB 程序时，在 Command Windows 窗口是否自动换行
 - ✓ Limit matrix display width to eighty columns 复选框用来选择是否限制矩阵的显示宽度为 80 列。
 - ✓ Enable up to 100 tab completions 用来设置允许制表符个数，缺省值为 100。
 - ✓ Command session scroll buffer size 用来设置命令文本显示的缓存大小。

另外，Command Windows 窗口的字体和颜色设置参见有关 Editor/Debugger 参数的设置内容。

✧ Edit (编辑) 菜单的内容

Edit 菜单的命令如图 2.4 所示，其意义如表 2.2 所示。

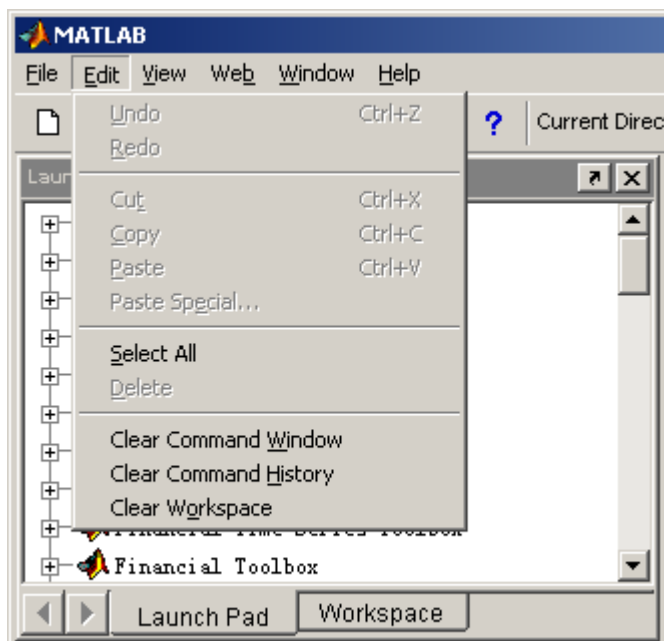


图 2.4 Edit (编辑) 菜单命令

表 2.2 Edit (编辑) 菜单的内容

菜单命令	功 能
<u>U</u> ndo Ctrl+Z	撤消上一步操作
<u>R</u> edo	重新执行上一步操作
<u>C</u> ut Ctrl+X	将选中的内容删除，放入剪贴板。
<u>C</u> opy Ctrl+C	将选中的内容放入剪贴板，但不删除所选的内容。
<u>P</u> aste Ctrl+V	将剪贴板的内容粘贴到指定位置。
Paste Special	选择性粘贴
<u>S</u> elect All	选择全部对象
<u>D</u> elete	删除对象
Clear Command <u>W</u> indow	清除 Command Window 窗口的内容
Clear Command <u>H</u> istory	清除 Command History 窗口的内容
Clear <u>W</u> orkspace	清除 Workspace 工作空间的内容

☆ View (视图) 菜单的内容

View 菜单的命令如图 2.5 所示，其意义如表 2.3 所示。

表 2.3 View (视图) 菜单的内容

菜单命令	功 能
Desktop <u>L</u> ayout	设置窗口的布局方式
<u>U</u> ndock Command Window	将 Command Window 设为最大化
Command Window/Command History/Current Directory/Workspace/Launch Pad	将相应窗口进行切换 (启动或消除相应窗口)
Help	启动帮助窗口
Current Directory Filter	设置当前目录筛选显示方式
Workspace View Options	设置工作区显示格式

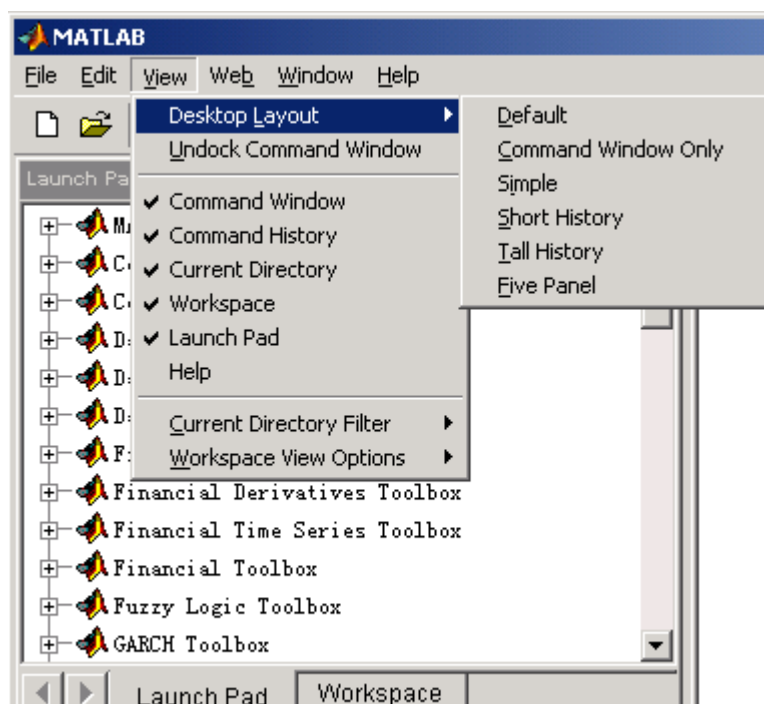


图 2.5 View (视图) 菜单命令

其中 Current Directory Filter 菜单的命令如图 2.6 所示，用户可以根据个人的喜好设置需要在 Current Directory 中所显示的文件或文件夹类型。此外，Workspace View Options 用于设置工作区 Workspace 变量的显示格式，如图 2.7 所示。可以根据个人喜好设置在 Workspace 工作区中的变量显示格式、排序方式。这些将在后续章节 (2.1.2.2) 中加以重点介绍。

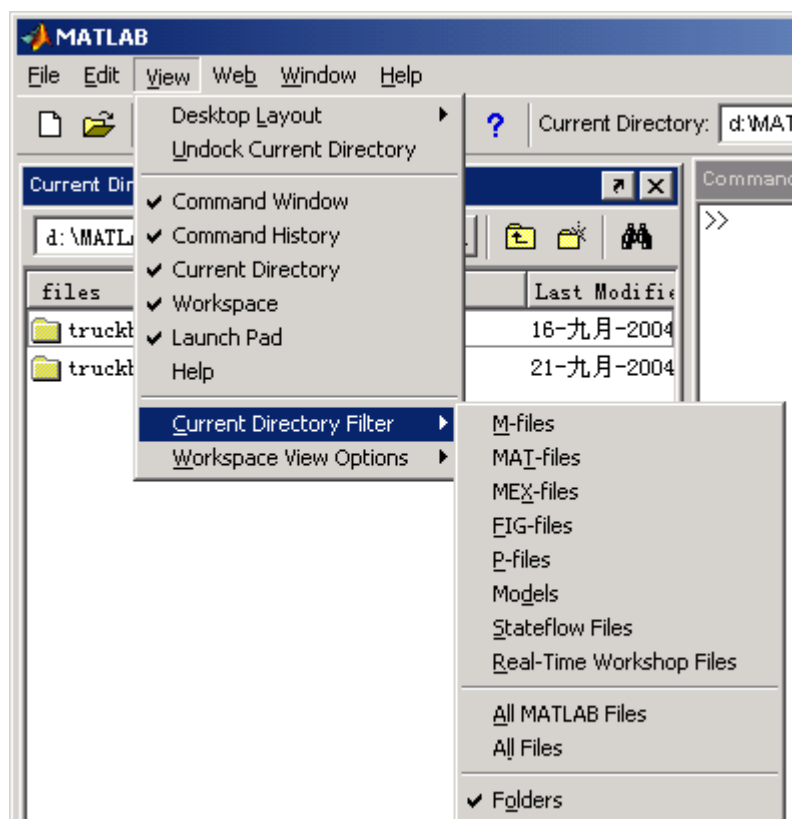


图 2.6 设置当前目录显示的文件类型

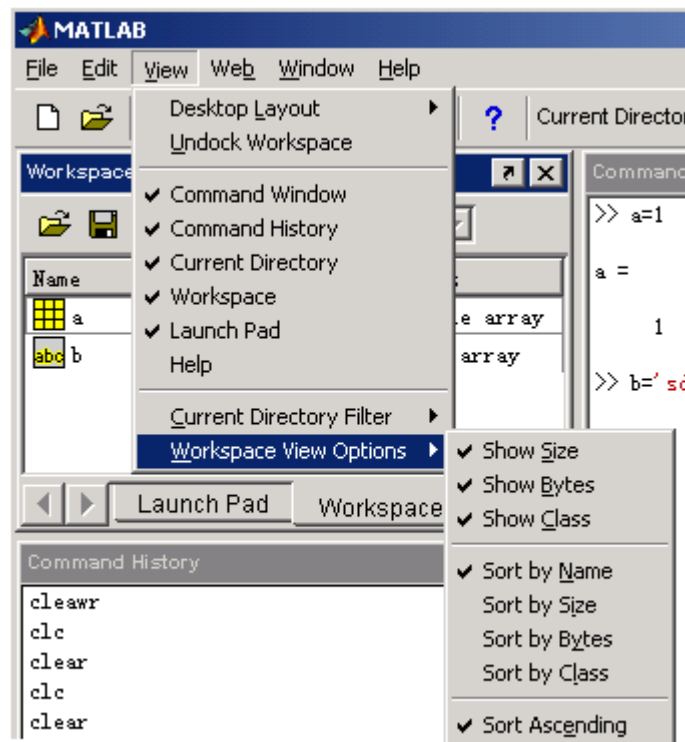
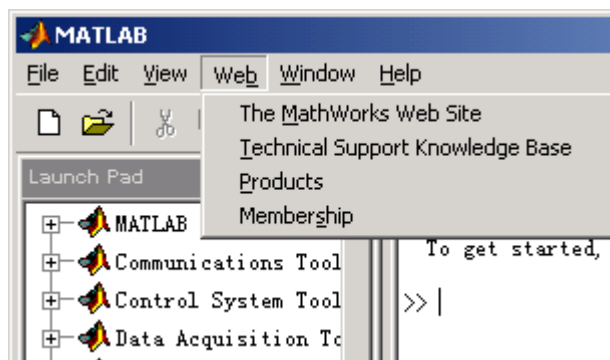


图 2.7 设置工作区变量显示格式

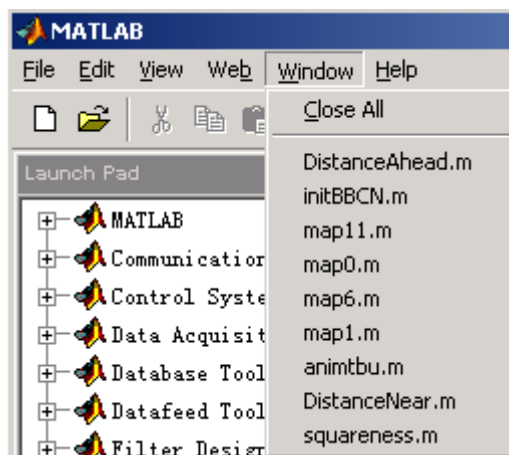
✧ Web（网络）菜单的内容

Web 菜单的命令主要用于链接相关的网址，包括 MathWorks 网址、技术支持、产品介绍与服务、会员注册等，这里就不一一赘述了。



✧ Window（窗口）菜单的内容

这个菜单应用于打开了多个 Matlab 窗口的情况，这时可以用他来在各个窗口之间切换。



◇ **Help（帮助）菜单的内容**

Help 菜单的命令如图 2.8 所示，其各个菜单项的意义如表 2.4 所示。当点击带有 Help 或 Using 字样的菜单项后，会出现帮助主窗口，如图 2.9 所示。当点击 Demos 后，出现演示主窗口，如图 2.10 所示。当点击 About MATLAB 后会出现关于 MATLAB 版权信息，图略。

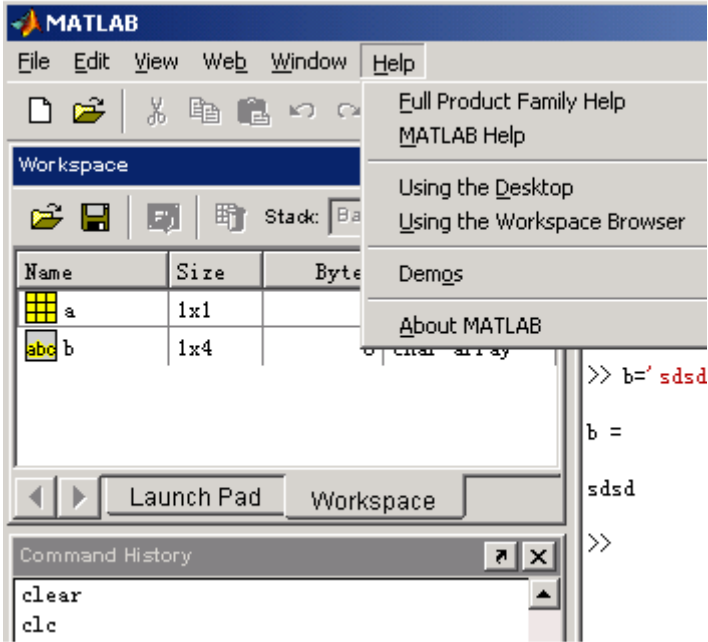


图 2.8 Help（帮助）菜单命令

表 2.4 帮助菜单项内容

菜单命令	功能
Full Product Family Help	完整的产品帮助
MATLAB Help	Matlab 帮助
Using the Desktop	使用桌面平台介绍
Using the Workspace Browser	使用工作区浏览器窗口帮助（只有工作区浏览器为焦点时才显示此菜单项）
Using the Launch Pad	使用启动盘帮助（只有启动托盘为焦点时才显示此菜单项）
Using the Command History	使用历史命令窗口帮助（只有历史命令窗口为焦点时才显示此菜单项）
Using the Current Directory Brower	使用当前目录浏览器帮助（只有当前目录浏览器为焦点时才显示此菜单项）
Using the Command Window	使用命令窗口帮助（只有命令窗口为焦点时才显示此菜单项）
Demos	启动演示窗口
About MATLAB	关于 MATLAB

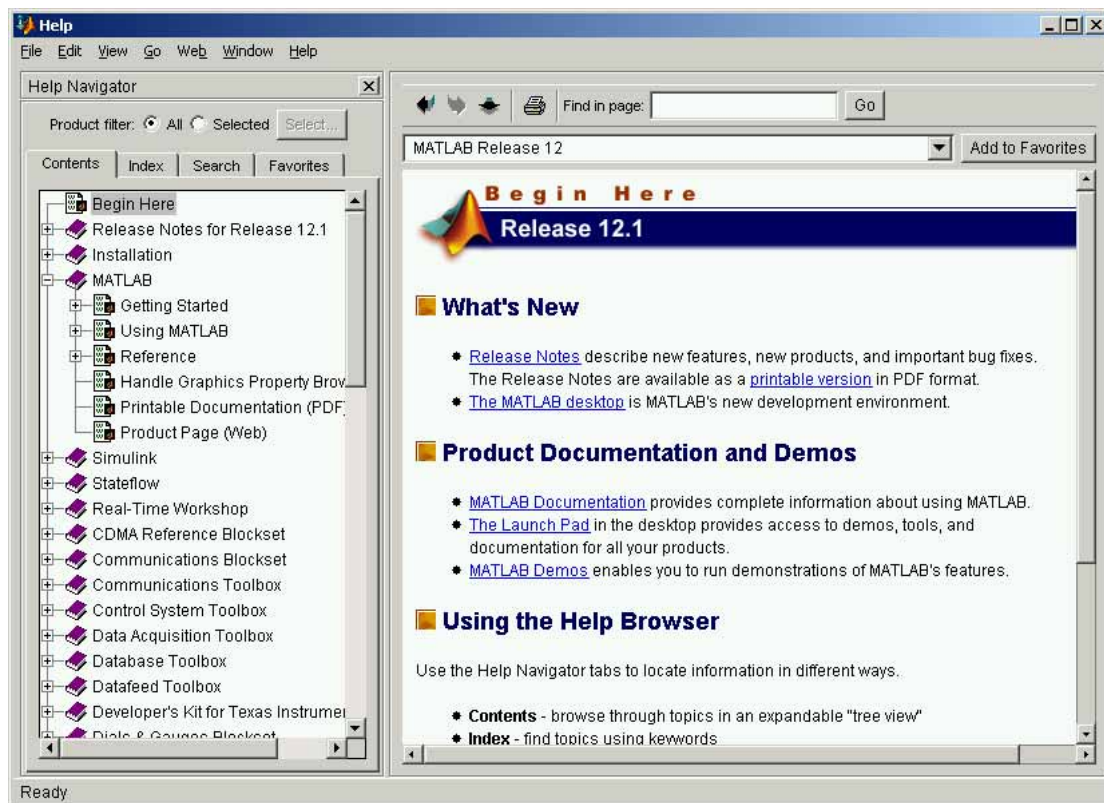


图 2.9 帮助主窗口

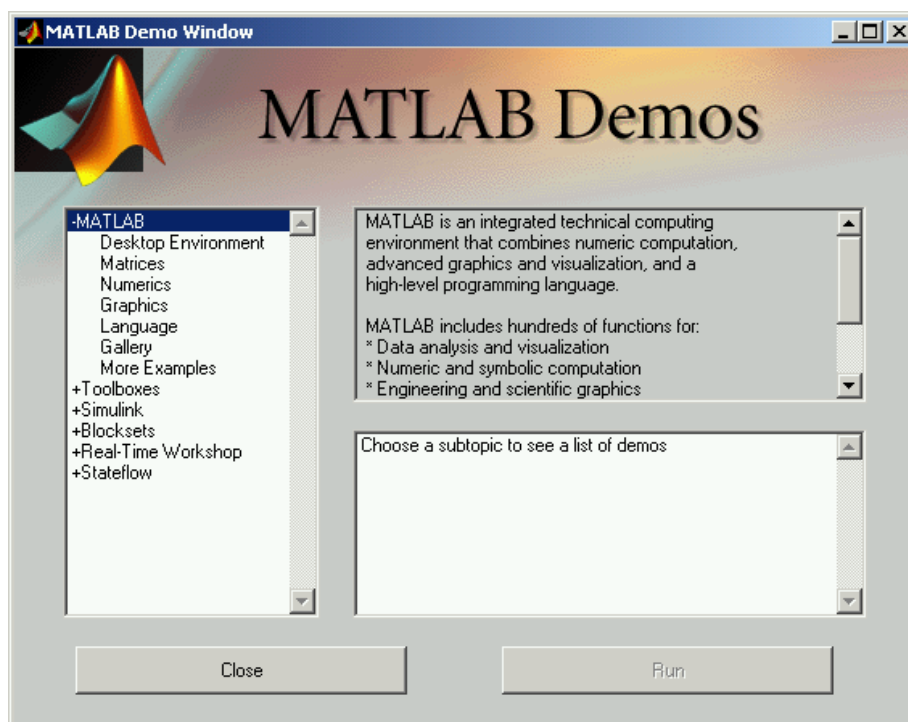


图 2.10 演示主窗口

2.2 若干通用操作命令

除了通过菜单命令对工作窗口进行控制外，MATLAB 还提供了许多通过键盘输入的控

制命令。表 2.5 所示为 Matlab 工作窗口中的一些通用操作命令

表 2.5 通用操作命令

操作命令	功能
cd、chdir	改变当前工作目录。
clear	清除内存中的所有变量。
clc	擦除 Matlab 工作窗口中所有显示的内容。
clf	擦除当前窗口中的图形。
Dir、ls	列出指定目录下的文件和子目录清单。
disp	在运行中显示变量或文字内容。
echo on/off	控制运行文字指令是否显示。
pack	收集内存碎块以扩大内存空间。
quit	关闭并退出 Matlab。
type test	显示所指定的文件 test.m 的全部内容。
what	列出当前目录下所有的 M 文件。
delete test	删除文件 test.m。
which test	列出指定名字文件 test.m 所在的目录。
hold on/off	控制当前图形窗口对象是否被刷新。
exist	检查指定名字的变量或函数文件的存在性。
who	列出工作内存中的变量。
whose	列出工作内存中的变量名称以及细节

2.3 命令行的编辑

由于 Matlab 是一种交互式的语言，因此随时输入指令后即时给出运算结果成为它的主要工作方式之一。表 2.6 列出了控制光标位置以及对指令进行操作的一些常用操作键。

表 2.6 常用的操作键及其功能

键盘操作		作用	键盘操作		作用
↑	Ctrl+p	调出前一个命令行。	Home	Ctrl+a	光标移至行首。
↓	Ctrl+n	调出后一个命令行。	End	Ctrl+e	光标移至行尾。
←	Ctrl+b	光标左移一个字符。	Esc	Ctrl+u	清除当前行。
→	Ctrl+f	光标右移一个字符。	Del	Ctrl+d	清除光标所在位置后的字符。
Ctrl+←	Ctrl+r	光标右移一个单词。	BackSpace	Ctrl+h	清除光标所在位置前的字符。
Ctrl+→	Ctrl+l	光标左移一个单词。		Ctrl+k	删至行尾。

三．桌面平台窗口简介

Matlab6.1 版本以后，新增加了几个停靠窗口，它们分别是：Launch Pad 窗口、Workspace 窗口、Command History 窗口、Current Directory 窗口、Command Windows 窗口。下面依次介绍各个窗口的作用。

✧ Launch Pad 窗口

用户可以在 Launch Pad 窗口中启动某个工具箱的应该程序, 单击 Launch Pad 窗口中的  按钮后, Launch Pad 窗口就脱离停靠主窗口如图 2.11 所示。通过 Launch Pad, 可以启动各个工具箱的帮助、Demos (演示窗口) 和其他相关的文件或者应用程序, 这是一个非常好的工具。通过它, 用户可以很方便的从事自己的工作, 比如要启动 Image Processing Toolbox (图形处理工具箱) 的 Demos, 双击该项目即可。

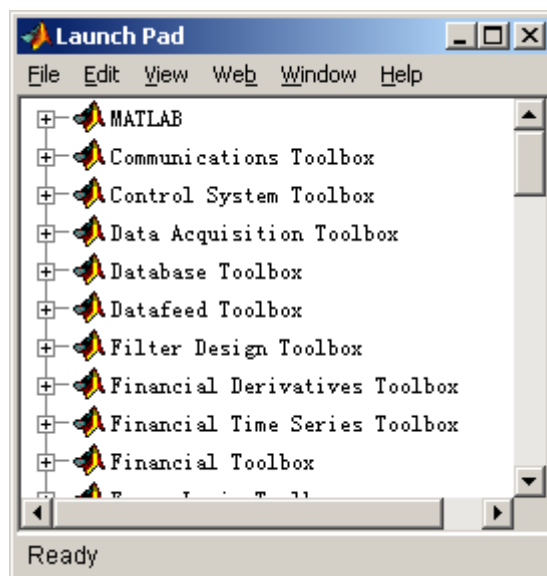


图 2.11 Launch Pad 脱离停靠 Matlab 主窗口

Launch Pad 窗口菜单与 Command Window 窗口菜单完全相同, 参见 2.1.1 一节的内容。

✧ Workspace 窗口

旧版本的 Workspace 是一个对话框, 可操作性差, 6.0 版本以后 Workspace 作为一个独立的窗口, 单击 Workspace 窗口中的  按钮后, Workspace 窗口就脱离停靠主窗口, 如图 2.12 所示。Workspace 窗口菜单与 Command Window 窗口中的菜单类似 (参见 2.1.2.1 菜单命令一节的内容), 只是在 View 菜单下增加了 Workspace View Option 命令, 如图 2.7 所示。

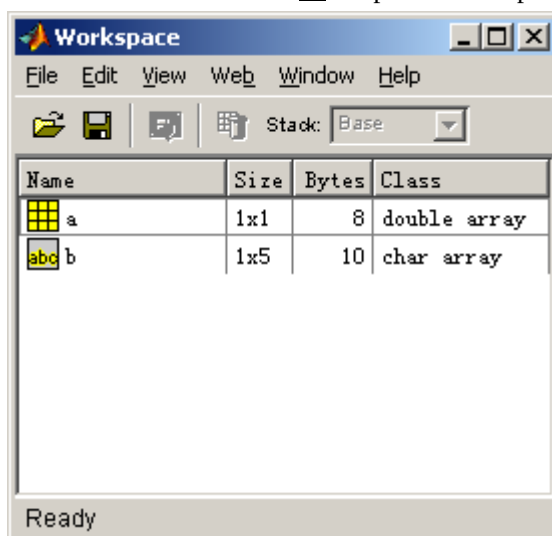


图 2.12 Workspace 脱离停靠 Matlab 主窗口

● 变量显示属性设置

View 菜单项下的 Workspace View Options 命令可以对 Workspace 窗口中变量的显示属性进行设置, 如图 2.7 所示, 各个命令的主要功能如表 2.7 所示。

表 2.7 变量的显示属性

菜单命令	功能	菜单命令	功能
Show Size	显示大小	Sort by Size	按照大小排序
Show Bytes	显示字节数	Sort by Bytes	按照字节数排序
Show Class	显示数据类型	Sort by Class	按照数据类型排序
Sort by Name	按照名称排序	Sort Ascending	按照升序排序

- 查看与清除工作空间中的变量

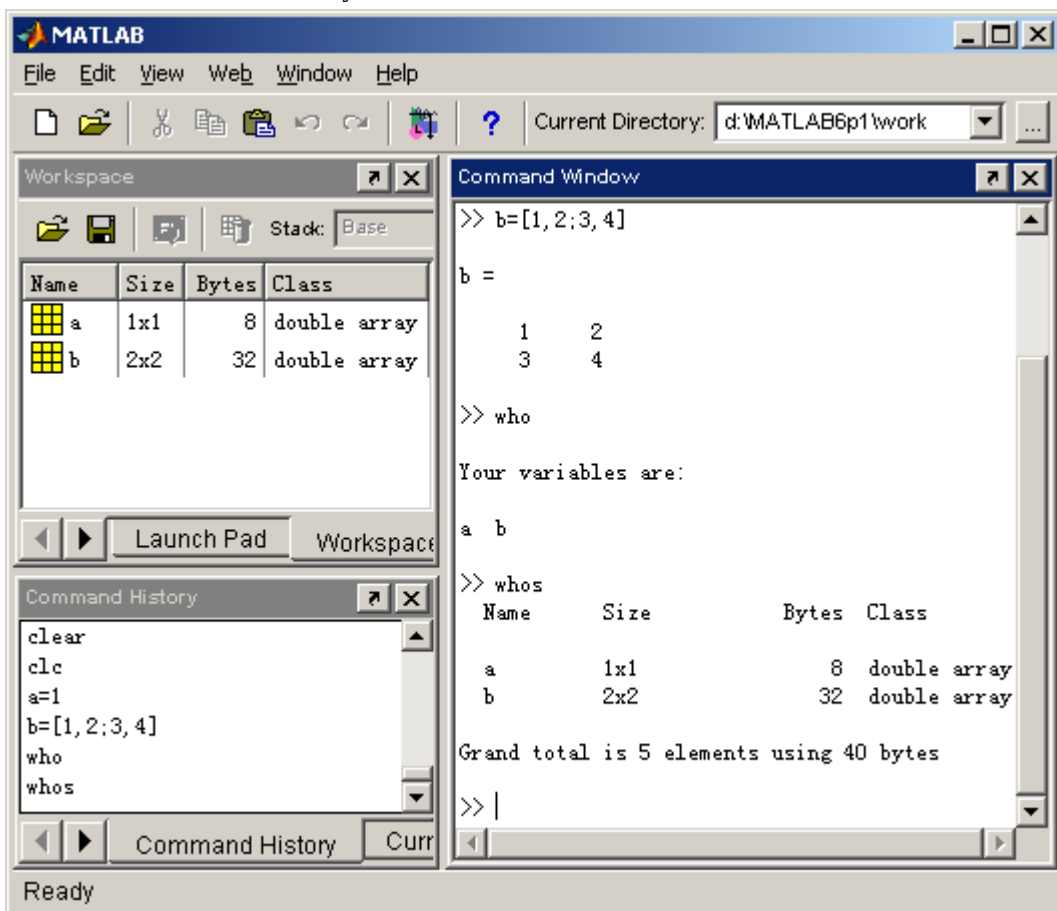
MATLAB 的工作空间包含一组变量（或命名的数组），用户可以在 Command Window 窗口对这些变量进行操作。用户可以用 `who` 或 `whos` 命令查看当前工作空间的内容。如图 2.13 所示为用 `who` 以及 `whos` 命令列出工作空间的所有变量。`who` 与 `whos` 命令的区别是 `who` 仅仅列出工作内存中的变量，而 `whos` 则可列出工作内存中的变量名称以及细节。

选择 `Edit→Clear+[窗口]` 命令可以清除相应窗口中的内容，其具体内容见表 2.2。如果选择 `Edit→Clear Workspace` 命令，则清除所有变量，选择该命令之后，系统会弹出一个提示对话框，让用户确认是否清除，因为清除后就无法恢复了。

通过工作空间浏览器（即 Workspace 窗口）也可以直接查看当前工作空间中的所有变量，如图 2.12 或 2.13 所示。其中列出了变量名称、大小、字节数和变量类型，与 `whos` 命令所列出的信息相同。选择其中一个变量，按 `Delete` 键可删除这个变量，也可以点击  按钮进行该操作，或者选择 `Edit→Delete` 命令。

- 数组内的显示和编辑

双击工作空间浏览器中的变量，如二维数组 `b`，系统会弹出 Array Editor（数组编辑器）窗口，也可以单击  按钮。Array Editor 窗口中显示了二维数组 `b` 的内容，如图 2.14 所示。

图 2.13 用 `who` 以及 `whos` 命令列出工作空间总的变量

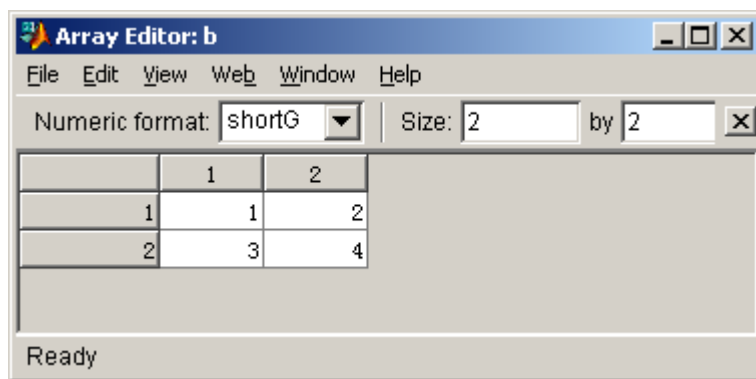


图 2.14 Array Editor（数组编辑器）窗口

用户可以在 Array Editor 窗口中对数组的大小以及每个元素进行编辑，但不可以改变数组的类型，如在数值型数组中输入字符串系统会自动给出错误提示。

- 保存工作空间

MATLAB 允许用户保存当前的工作空间，随时调用已保存的工作空间，并可以进行文本数据文件的输入和输出。

使用 save 命令可以把工作空间保存在二进制的 MAT 文件中，以后可以使用 load 命令调用 MAT 文件。例如要保存整个工作空间的内容，可以在 Command Window 窗口中使用以下命令：save myworks，则把当前工作空间的内容保存在当前目录下的 myworks.mat 文件中。如果仅仅要保存工作空间中的几个变量，可以在保存的文件名后加上要保存的变量列表，例如，把当前工作空间中的变量 var1，var2 保存在 myworks.mat 文件中，可以使用以下命令：save myworks var1 var2。另外，选择 File→Save Workspace As 命令也可以保存工作空间。

在保存工作空间时可以指定一些参数，控制保存文件的格式。各个参数如表 2.8 所示。例如，要把当前工作空间中的所有变量以 ASCII 格式保存到 data1 文件中，可以在 Command Window 窗口中输入如下命令：

表 2.8 保存工作空间的参数

参数	功能
.mat	使用二进制 MAT 文件格式（系统默认的格式）
.ascii	使用 8 位 ASCII 格式
.ascii .double	使用 16 位 ASCII 格式
.ascii .double .tabs	使用制表符分隔数组元素
.v4	保存为与 MATLAB 4 兼容的格式
.append	向已经存在的 MAT 文件中添加数据

注意：使用参数 v4 时只能 MATLAB 6 中与 MATLAB 4.x 兼容的数据类型，因此像结构体、单元数组和对象不被保存，而且保存的文件名应为 MATLAB 4.x 支持的文件名格式。使用 ASCII 格式保存工作空间内容时，每次仅保存一个变量。如果多于一个变量，仍可以创建 ASCII 保存变量，但是不能使用 load 命令加载已保存的数据。

- 加载工作空间

用 save 命令保存的工作空间文件，可以用 load 命令加载。例如：load myworks。如果当前工作空间与加载的工作空间具有相同的变量，则后加载的变量将自动覆盖与原工作空间中同名的变量。如果保存 MAT 文件使用了其他扩展名，如“worksp.s”。加载时则要使用参数“.mat”指定其为 MAT 格式的文件，并且一定要加上文件的扩展名。例如：

```
load worksp.s.mat
```

在主窗口、Command Window 窗口和 Workspace 窗口中选择 File→Import Data 命令，也可以加载工作空间。加载时系统会弹出加载工作空间向导，如图 2.15 所示。通过此向导，

可以从 MAT 数据文件中选择加载到工作空间中的变量。

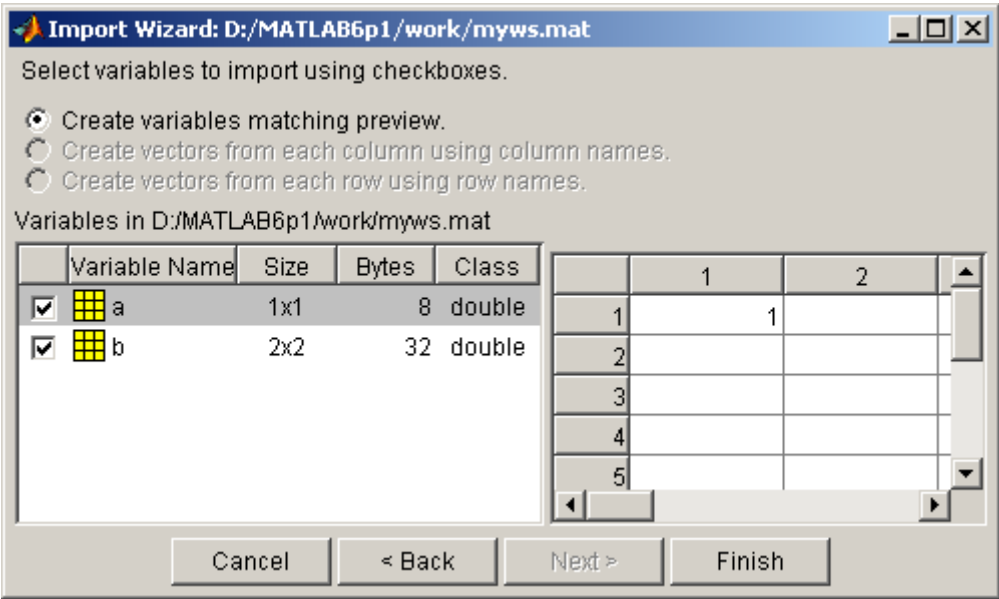


图 2.15 加载工作空间向导

使用 load 命令也可以加载 ASCOOII 数据文件，把文件的内容读取到一个与文件名相同的变量中，如图 2.16 所示。

图 2.16 中各个命令的意义如下所示：

b=[1,2;3,4];	定义变量 b
save var1.dat -ascii	以 8 位格式保存当前的工作空间
clear	清除当前工作空间中的所有变量
load var1.dat	加载已保存的 ASCII 格式
whos	列出当前工作空间中的变量
var1	给出变量把 bar1 的数值

从图 2.16 的结果可以看出，以前定义的变量 b 不在当前的工作空间中，而当前工作空间中的变量名为 var1，是保存工作空间的文件名。此变量的大小、类型和字节数与变量 b 完全相同。

● 文件名和变量名的特殊用法

如果文件名和变量名保存在字符串变量中，在保存或加载工作空间时，则可以使用字符串变量，同时可以把 save 和 load 命令理解为一般的函数，参数的顺序与命令中参数的顺序相同。例如下面的一段命令：

```
a=1
b=1
save('myfile','a','b')
A='myfile'
Load(A) %使用字符型变量 A 作为函数 load 的参数。
```

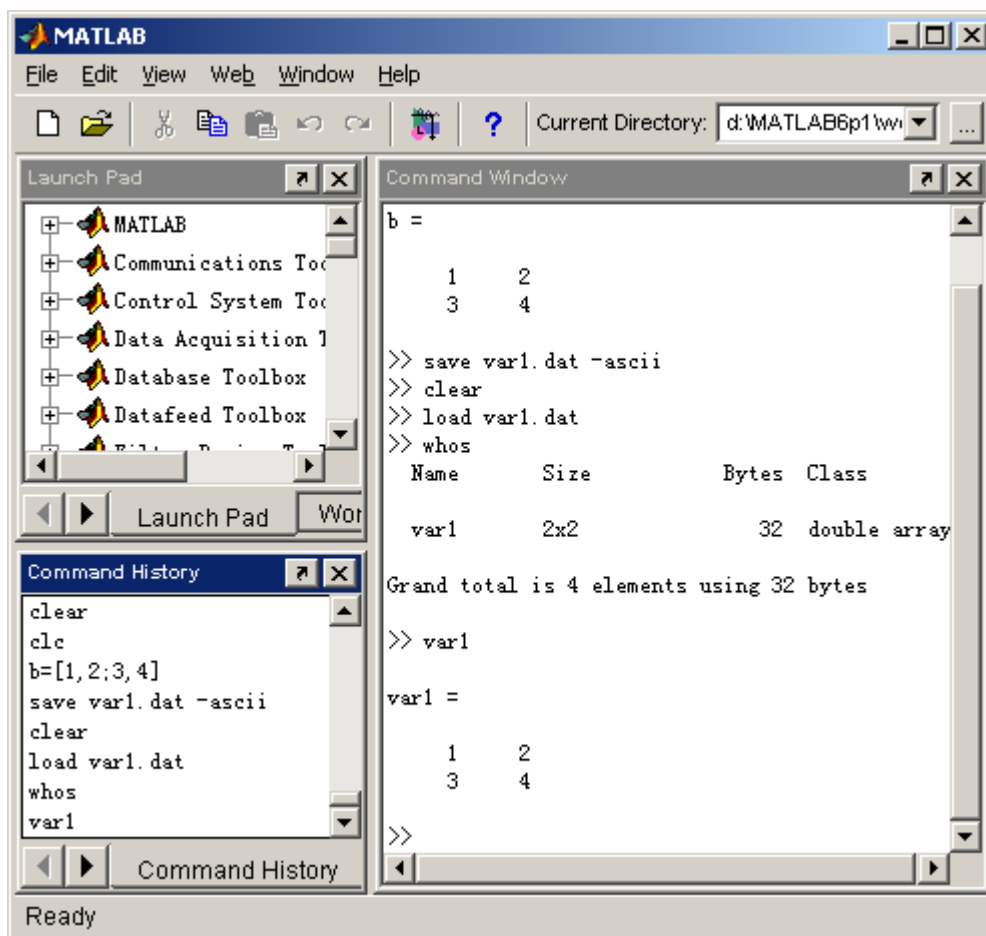


图 2.16 加载 ASCII 数据文件

等效于以下命令：

save myfile a b

load myfile

同样也等效于以下命令：

A='myfile'

B='a'

C='b'

%定义 3 个字符变量保存文件名和变量名

save(A,B,C)

%使用 save 命令的函数方式，保存字符变量 A，B 和 C。

load(A)

命令 save 和 load 都可以使用通配符 “*” 搜索相似的变量名。

例如：save myworks a*，把当前工作空间中所有以 a 开头的变量保存在文件 myworksp.mat 中。又如：load myworksp aa*bb，从文件 myworksp.mat 中加载所有变量名中前两个字符是 aa 的并且最后两个字符是 bb 的变量。通配符的位置可以是任意字符或字符串。

✧ Command History 窗口

Command History 窗口主要显示已经执行过的命令。Matlab 每次启动时，Command History 窗口会自动记录启动的时间，并将 Command History 窗口中执行的命令记录下来。一方面便于查找，另一方面可以再次调用这些命令，如图 2.17 所示。双击 Command History 窗口中的命令，该操作等效于在 Command History 窗口输入该命令。

Command History 窗口菜单与 Command Window 窗口中的菜单完全相同，参见 2.1.2 菜单命令一节的内容。

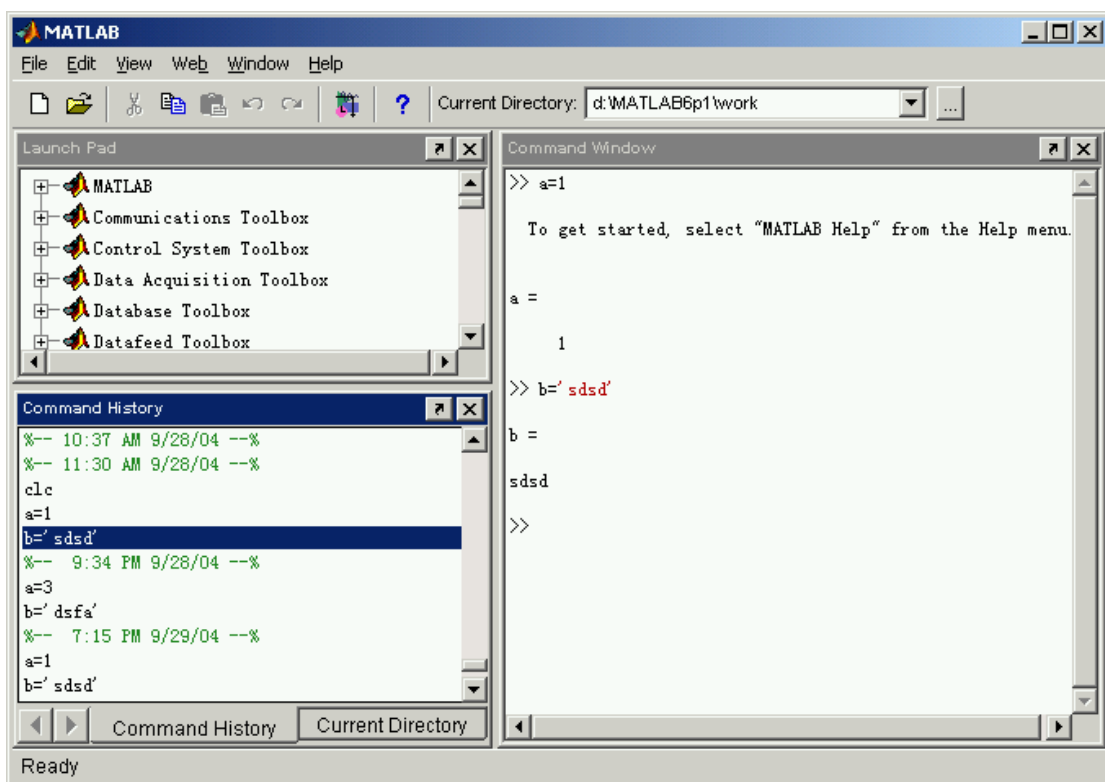


图 2.17 调用 Command History 窗口的命令

☆ Current Directory 窗口

Current Directory 窗口菜单与 Command Window 窗口类似,只是在 View 中增加了 Current Directory 命令,如图 2.6 所示。该菜单用于设置在 Current Directory 窗口中显示的文件类型。Matlab 文件类型如表 2.9 所示。

Current Directory 窗口主要显示的是当前在什么路径下进行工作,包括文件的保存等都是在当前路径下实现的。用户也可以选择 File→Set path 命令设置当前路径,如图 2.18 所示。图中的各个按钮功能如表 2.10 所示。

表 2.9MATLAB 文件类型

文件类型	说明	扩展名
M-file	M 文件	.m
MAT-file	数据文件	.mat
MEX-file	MEX 文件	.mex 或.dll
FIG-file	图形文件	.fig
P-file	P 文件	.p
Models	模型文件	.mdl

表 2.10 Set Path 对话框中的按钮及其功能

按钮	功能
Add Folder	选择目录添加至 Matlab searchpath (搜索路径) 中
Add with Subfolders	将选中的目录路径的子目录也包含在搜索路径中
Move to Top	将搜索路径中选中的目录路径移至顶部
Move Up	将搜索路径中选中的目录路径向上移
Remove	清除搜索路径中选中的目录路径
Move Down	将搜索路径中选中的目录路径向下移
Move to Buttom	将搜索路径中选中的目录路径移至底部

Save	将设定好的搜索路径保存
Close	关闭 Set path 窗口
Revert	撤销先前操作，恢复原状
Default	系统默认的搜索路径
Help	打开帮助窗口

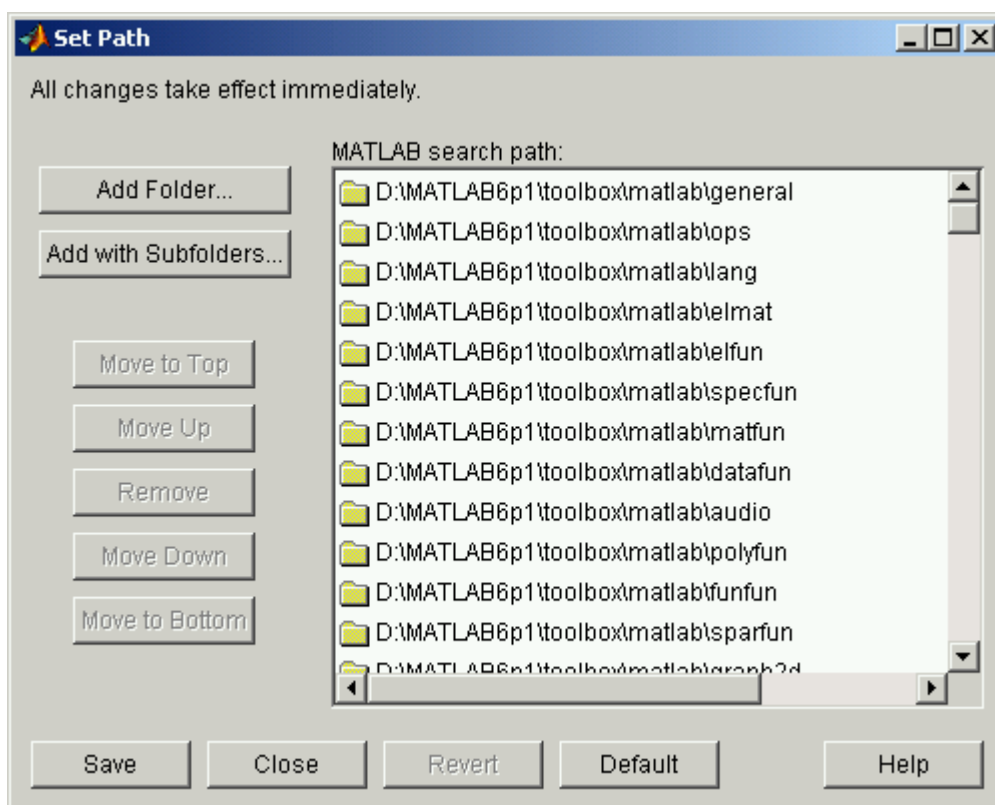


图 2.18 Set Path 对话框

Matlab 采用路径搜索的方法来查找组织在文件系统中扩展名为.m 的 M 文件，常用的命令文件组织在 Matlab 文件夹中，其他 M 文件组织在各种工具箱目录中。在 Command Window 窗口中输入一个字符串 `matsear` 时，将按以下的顺序开始搜索：

- (1) 把 `matsear` 作为一个变量进行搜索，在当前工作空间中查找变量 `matsear`。
- (2) 把 `matsear` 作为一个内置函数进行搜索，查找内置函数 `matsear` 并执行。
- (3) 查找当前命令中的 `matsear.m` 文件。
- (4) 查找当前搜索路径中的 M 文件 `matsear.m`

此搜索的顺序只是一般情况下的顺序，而实际上的搜索规则还要考虑到私有函数、子函数和面向对象函数的范围限制，因此会更加复杂。但此简单的搜索顺序，对普通的 M 文件是非常准确的。

如果在搜索路径中存在同名函数，则仅可发现搜索建立中的第 1 个函数，而其他同名的函数不被执行。使用下面的一组命令可以对当前的搜索路径进行操作。

- `Path`：不加任何参数，显示当前搜索路径；
- `Path+[路径名称]`：设置当前的搜索路径，以前的搜索路径无效。
- `addpath/sys/hhf` 或 `path (path, ' ')`：向当前搜索路径中添加目录 `sys/hhf`，后者为函数形式。
- `rmpath/sys/hhf`：取消当前搜索路径中的目录 `sys/hhf`。

- pathtool：打开如图 2.18 所示的 Set Path 对话框设置搜索路径。

MATLAB 将默认的搜索路径保存在 pathdef.m 文件中，MATLAB 启动时将自动执行。此文件保存在 \MATLAB\toolbox\local 目录下，用户可以使用文本编辑器直接对其进行编辑。例如在 Command Window 窗口中键入：Edit pathdef.m 即可对该文件进行编辑。

四. M 文件的编辑调试环境

MATLAB 的程序文件和脚本文件通常保存为后缀为 “. m” 的文件，我们就称之为 M 文件。编辑 M 文件当然可以用其他的文本编辑器，要启动 MATLAB 的 M 文件编辑器和调试器，可在命令窗口中输入 Edit 命令，也可以选择 File 菜单中的 New→M-file 选项命令，或者单击图标按钮。编辑器和调试器如图 2.19 所示。

4.1 工具条简介

工具条位于菜单的下方，参见图 2.20，它的图标以及功能参见表 2.11 所示。

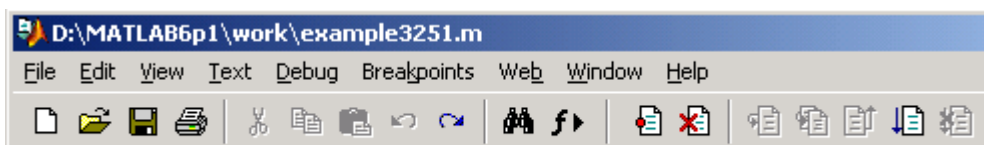


图 2.20 工具条

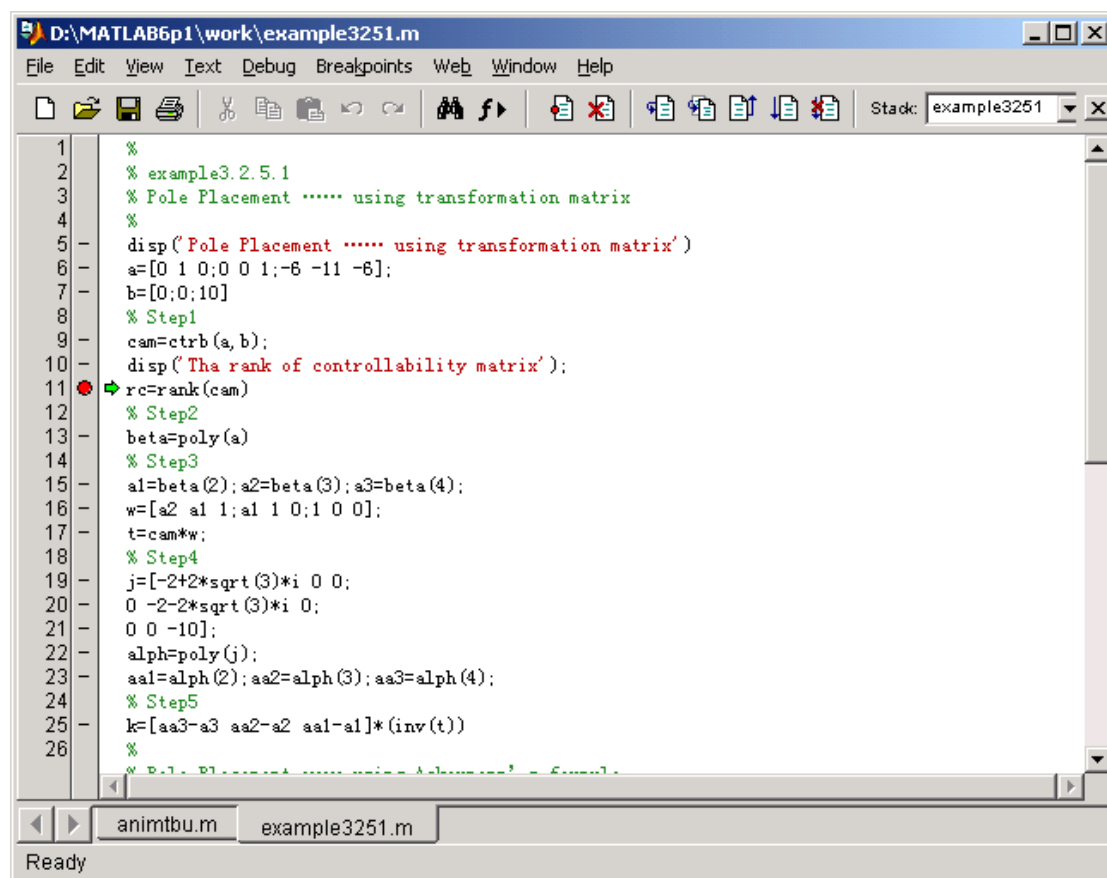


图 2.19 M 文件的编辑器和调试器

表 2.11 工具条的功能简介

图标	功能	图标	功能
	查找并代替字符串		转入被调用的函数，单步执行
	显示函数		从被调用的函数快速转出
	设置/清除断点		继续调试
	清除所有断点		退出程序调试
	继续执行		

4.2 M 文件编辑器和调试器简介

M 文件编辑器和调试器的 File 菜单和 Edit 菜单与一般的文本文件编辑器相似。File 菜单和 Edit 菜单的选项如图 2.21 和图 2.22 所示。File 菜单以及 Edit 菜单选项的功能如表 2.12 以及 2.13 所示，View 菜单选项的内容与主窗口的 View 菜单选项的内容类似，参见前面相应的章节。

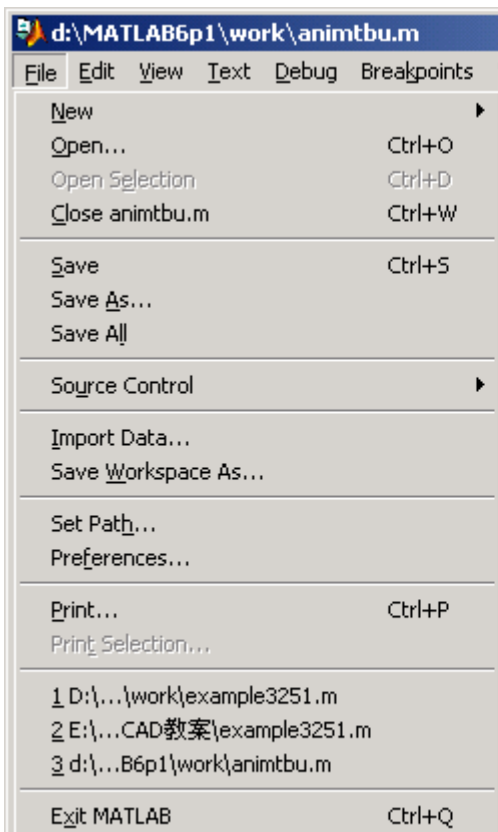
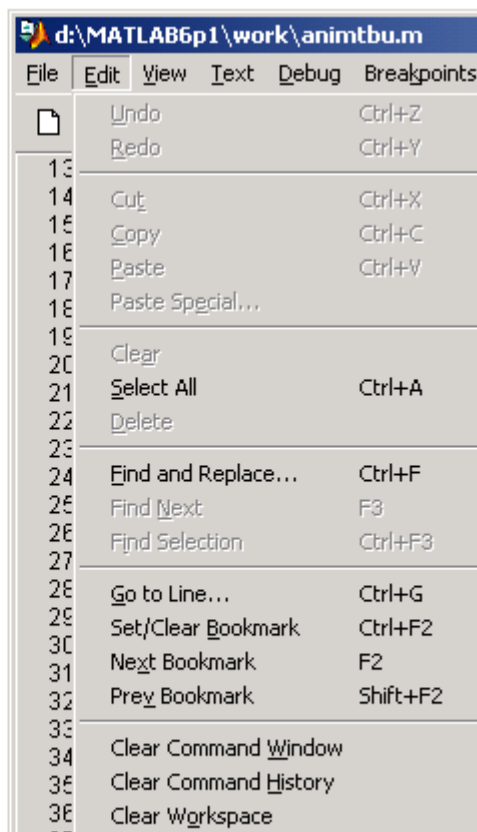


图 2.21 File菜单选项



2.22 Edit菜单选项

表 2.12 File 菜单选项功能

菜单项	功能
New	新建 M 文件
Open	打开 M 文件
Open Seledon	仅仅打开所选中的对象
Close	关闭文件
Save	保存文件
SaveAs	另存为
Save All	保存所有 M 文件
Source Control	该子菜单项允许用户检查用户的源控制系统
Import Data	从外部文件输入数据
Save Workspace As	将当前工作空间另存为其他文件
Set Path	设置 MATLAB 搜索路径
Preferences	设置编辑器/调试器的参数
Print	打印输出
Print selection	打印所选中的文本或其他对象
Exit MATLAB	退出 MATLAB 系统

表 2.13 Edit 菜单项功能

菜单项	功能
Undo	取消上一步操作
Redo	重新执行上一步操作
Cut	删除
Copy	复制
Paste	粘贴
Paste special	选择性粘贴
Clear	清除对象
Select All	全部选取
Delete	删除对象
Find and Replace	查找并替代对象
Find Next	否找下一个对象
Find Selection	查找所选择的对象
Goto Line	跳转到指定的行

Text 菜单中的命令选项如图 2.23 所示，其功能如下：

- Comment，注释程序行，执行该命令后，则鼠标所在的程序行无效。
- Uncomment，取消程序行注释，执行该命令后，则鼠标所在的程序行有效。
- Increase Indent，增加文本的缩进。
- Decrease Indent，减少文本的缩进。
- Balance Delimiters，平衡分界符。
- Smart Indent，智能缩进，即使用系统的设定对文本自动缩进处理。
- Evaluate Selection，计算所选部分表达式的值。

Debug 菜单选项如图 2.24 所示。Debug 菜单中各选项的意义如下：

- Step，继续调试过程。
- Step In，运行调试的程序遇到断点后，选择此项可以转入被调用的函数或程序并可对

具调试。

- Step out, 快速调试, 执行此选项, 可从设置断点函数中快速转出 v 继续体继续调试。
- Continue, 运行调试的程序遇到断点后, 选择此项继续执行。
- Go Until Cursor, 执行到光标所在处。
- Exit Debut Mode, 退出程序调试。

Breakpoint 菜单项如图 2.25 所示, Breakpoint 菜单项各选项的意义如下:

- Set/Clear Breakpoint, 设置/清除断点, 该命令功能与按钮相同。
- Clear all Breakpoint 清除所有断点, 该命令功能与按钮相同。
- Stop If Error, 发生错误时停止程序执行。
- Stop If Warning, 出现警告时停止程序执行。
- Stop If NaN or inf, 遇到非数值(Non-a-Number)或无穷大 (Infinity) 时停止程序执行,

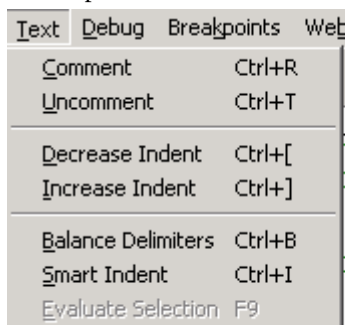


图2.23 Text菜单选项

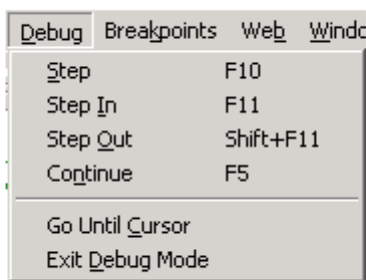


图2.24 Debug菜单选项

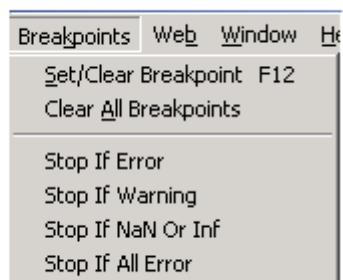


图2.25 Breakpoint菜单选项

实验一. MATLAB 语言基础

一. MATLAB 的基本计算功能

1. 变量与数值显示格式

(1) 建立若干个变量, 用 who(s)命令查询变量名, 用 clear 清除变量名, 用 save 保存变量为数据文件, 用 load 从数据文件中读入变量。

变量规则:

- ✧ 变量名必须以字母开头(不能超过 19 个字符), 之后可以是任意字母、数字, 或下划线(_);
- ✧ 变量区分字母的大小写, 同一名字的大写与小写被视为两个不同的变量;
- ✧ 变量中不能含有标点符号。

Matlab 中含有自己的一些特殊变量, 如表 2.1 所示, 显示 Matlab 中特殊变量的数值, 并将其保存在文件中。

表 2.1 Matlab 中含有的特殊变量

特殊变量	取 值	特殊变量	取 值
ans	缺省的结构变量名	i,j	$\sqrt{-1}$
pi	圆周率	Flops	浮点运算数
Nargin	函数输入变量数目	Nargout	函数输出变量数目
realmax	最大的可用正实数	realmin	最小的可用正实数
inf	无穷大 如 1/0	NaN	不定量 如 0/0
eps	计算机的最小数		

(2) 查看某数值的显示格式, 缺省情况下整数用整数显示; 实数的小数点后有 4 位。

例 1.1.1: 用 Matlab 显示 430/12 的显示结果如表 2.2 所示:

表 2.2 430/12 不同的显示结果

Format 命令改变	430/12 的显示结果	说明
Format short	35.833 3	缺省显示
Format long	35.833 333 333 333 34	16 位
Format short e	35.8 3e+01	5 位加指数
Format long e	3.583 333 333 333 334e+01	16 位加指数
Format hex	4041eaaaaaaaaaab	16 进制
Format bank	35.83	2 个十进制位
Format +	+	正、负、零
Format rat	215/6	有理数近似

2. 学习使用 MATLAB 的基本数学函数

<code>abs(x)</code>	纯量的绝对值或向量的长度
<code>angle(z)</code>	复数 z 的相角
<code>sqrt(x)</code>	开平方
<code>real(z)</code>	复数 z 的实部
<code>imag(z)</code>	复数 z 的虚部
<code>conj(z)</code>	复数 z 的共轭复数
<code>round(x)</code>	四舍五入到最近的整数
<code>fix(x)</code>	无论正负，舍去小数至最近的整数
<code>floor(x)</code>	舍去正小数至最近的整数
<code>ceil(x)</code>	加入正小数至最近的整数
<code>rat(x)</code>	将实数 x 化为分数表示
<code>rats(x)</code>	将实数 x 化为多项分数展开
<code>sign(x)</code>	符号函数 -1 ; 0 ; +1。
<code>rem(x,y)</code>	求 x 除以 y 的余数
<code>gcd(x,y)</code>	整数 x 和 y 的最大公因数
<code>lcm(x,y)</code>	整数 x 和 y 的最小公倍数
<code>Exp(x)</code>	自然指数
<code>Pow2(x)</code>	2 的指数
<code>Log(x)</code>	以 e 为底的对数 $\ln(x)$
<code>Log2(x)</code>	以 2 为底的对数 $\log_2(x)$
<code>Log10(x)</code>	以 10 为底的对数 $\log_{10}(x)$

3. 学习使用 MATLAB 的常用三角函数

<code>sin(x)</code>	正弦函数
<code>cos(z)</code>	余弦函数
<code>tan(x)</code>	正切函数
<code>asin(x)</code>	反正弦函数
<code>acos(z)</code>	反余弦函数
<code>atan(z)</code>	反正切函数
<code>atan2(x,y)</code>	四象限反正切
<code>sinh(x)</code>	双曲正弦函数
<code>cosh(z)</code>	双曲余弦函数
<code>tanh(x)</code>	双曲正切函数
<code>asinh(x)</code>	反双曲正弦函数
<code>acosh(z)</code>	反双曲余弦函数
<code>atanh(z)</code>	反双曲正切函数

二. MATLAB 矩阵和数组的创建和保存

1. MATLAB 的矩阵输入

❖直接输入创建矩阵

例 1.2.1 用指令产生数值矩阵

```
A = [] %生成一个空矩阵
x=9;y=pi/6; % 定义 x,y 变量
A=[3 5 sin(y)
    cos(y) x^2 7
    x/2 5 1] % 产生矩阵 A
A=
    3.000    5.0000    0.5000
    0.8660   81.0000    7.0000
    1.5000    5.0000    1.0000
```

说明：%作为 Matlab 注释的开始标志，以后的文字不影响计算的过程。

矩阵中的元素可以用它的行和列数表示，例如 A(2,3)表示矩阵 A 的第二行第三列的元素 7。对于一个已经建立的矩阵，Matlab 有许多方法修改它的内容。

例 1.2.2 矩阵元素的修改

```
A(3,3)=0
A=
    3.0000    5.0000    0.5000
    0.8660    81.0000    7.0000
    4.5000    5.0000     0
说明：A(3,3)=0 将矩阵 A 的第三行第三列的元素变成 0。
A(2,6)=1
A=
    3.0000    5.0000    0.5000     0     0     0
    0.8660    81.0000    7.0000     0     0    1.0000
    4.5000    5.0000     0     0     0     0
```

A(2,6)=1 在矩阵的第二行第六列置为 1。由于矩阵 A 没有 6 列，Matlab 系统自动增加 A 的列数以适应需要，并将其他位置置为 0。

前面谈到复数的时候，提到矩阵中也可以使用复数，下面看看如何用复数建立矩阵。复数矩阵有两种生成方式：

例 1.2.3 复数矩阵的建立和输入

第一种方式

```
a=2.7;b=13/25;
C=[1,2*a+i*b,b*sqrt(a); sin(pi/4),a+5*b,3.5+1]
```

显示 C 矩阵的内容如下

```
C=
    1.0000          5.4000 + 0.5200i    0.8544
```

0.7071

5.3000

4.5000

第 2 种方式

复数矩阵的输入还有一个特殊的方法,即用一个矩阵表示复数的实部,而用另一个矩阵表示复数的虚部。最后将两个矩阵相加,可以得到所需的复数矩阵。

```
R=[1 2 3;4 5 6], M=[11 12 13;14 15 16]
```

```
CN=R+i*M
```

显示 R, M, CN 矩阵的内容如下

```
R =
```

```
1     2     3
4     5     6
```

```
M =
```

```
11    12    13
14    15    16
```

```
CN =
```

```
1.0000 +11.0000i  2.0000 +12.0000i  3.0000 +13.0000i
4.0000 +14.0000i  5.0000 +15.0000i  6.0000 +16.0000i
```

例 1.2.4 符号矩阵的生成

在 MATLAB 中输入符号向量或者矩阵的方法和输入数值类型的向量或者矩阵在形式上很相像,只不过要用到符号矩阵定义函数 `sym`,或者是用到符号定义函数 `syms`,先定义一些必要的符号变量,再像定义普通矩阵一样输入符号矩阵。

1. 用命令 `sym` 定义矩阵:

这时的函数 `sym` 实际是在定义一个符号表达式,这时的符号矩阵中的元素可以是任何的符号或者是表达式,而且长度没有限制,只是将方括号置于用于创建符号表达式的单引号中。如下例:

```
sym_matrix = sym ('[a b c ; Jack, Help Me!, NO WAY!]', 's')
sym_matrix =
    [a          b          c]
    [Jack    Help Me!    NO WAY!]
sym_digits = sym ('[1 2 3 ; a b c ; sin (x) cos (y) tan (z) ]')
sym_digits =
    [1          2          3]
    [a          b          c]
    [sin (x) cos (y) tan (z) ]
```

2. 用命令 `syms` 定义矩阵

先定义矩阵中的每一个元素为一个符号变量,而后像普通矩阵一样输入符号矩阵。

```
syms a b c ;
M1 = sym ('Classical') ;
M2 = sym ('Jazz') ;
M3 = sym ('Blues')
syms_matrix = [a b c ; M1, M2, M3 ; int2str ([2 3 5]) ]
syms_matrix =
    [ a          b          c]
    [Classical  Jazz  Blues]
    [ 2          3          5]
```

把数值矩阵转化成相应的符号矩阵。

数值型和符号型在 MATLAB 中是不相同的，它们之间不能直接进行转化。MATLAB 提供了一个将数值型转化成符号型的命令，即 sym。

如下例：

```
Digit_Matrix = [1/3 sqrt(2) 3.4234; exp(0.23) log(29) 23^(-11.23)]
```

```
Syms_Matrix = sym(Digit_Matrix)
```

结果是：

```
Digit_Matrix =
```

```
0.3333    1.4142    3.4234
1.2586    3.3673    0.0000
```

```
Syms_Matrix =
```

```
[ 1/3, sqrt(2), 17117/5000]
[5668230535726899*2^(-52), 7582476122586655*2^(-51), 5174709270083729*2^(-103)]
```

注意：矩阵是用分数形式还是浮点形式表示的，将矩阵转化成符号矩阵后，都将以最接近原值的有理数形式表示或者是函数形式表示。

例 1.2.5 大矩阵的生成

对于大型矩阵，一般创建 M 文件，以便于修改：

用 M 文件创建大矩阵，文件名为 example.m

```
exm=[ 456    468    873     2   579    55
      21    687    54   488     8    13
      65   4567    88    98    21     5
      456    68  4589   654     5   987
      5488   10     9     6    33   77]
```

在 MATLAB 窗口输入：

```
example;
```

```
size(exm) %显示 exm 的大小
```

```
ans=
```

```
5 6 %表示 exm 有 5 行 6 列。
```

例 1.2.6 多维数组的创建，假设可以如下定义 A_1, A_2, A_3 矩阵： $A_1=[1,2,3;4,5,6;7,8,9]$; $A_2=A_1'$; $A_3=A_1-A_2$; 则通过下面最原始的方法就可以定义出一个三维数组 A_4 。

```
A4(:,:,1)=A1, A4(:,:,2)=A2, A4(:,:,3)=A3
```

```
A4(:,:,1) =
```

```
1     2     3
4     5     6
7     8     9
```

```
A4(:,:,2) =
```

```
1     4     7
2     5     8
3     6     9
```

```
A4(:,:,3) =
```

```
0    -2    -4
2     0    -2
4     2     0
```

MATLAB 语言提供了一个 cat() 函数来构造多维数组。该函数的调用格式如下：

$A = \text{cat}(n, A_1, A_2, \dots, A_m)$

其中 $n=1$ 和 $n=2$ 时分别构造 $[A_1; A_2; A_3; \dots; A_m]$ 和 $[A_1, A_2, \dots, A_m]$, 结果是二维数组, 而 $n=3$ 可以构造出三维数组。例如前面原始的命令可以由下面的简单函数调用语句取代。

```
A1=[1,2,3;4,5,6;7,8,9];A2=A1';A3=A1-A2;
```

```
A4=cat(3,A1,A2,A3)
```

```
A4(:, :, 1) =
```

```
1     2     3
4     5     6
7     8     9
```

```
A4(:, :, 2) =
```

```
1     4     7
2     5     8
3     6     9
```

```
A4(:, :, 3) =
```

```
0    -2    -4
2     0    -2
4     2     0
```

比较下面两个语句获得的二维矩阵,

```
A5=cat(2, A1, A2, A3)
```

```
A5 ( ; , 1) =
```

```
1     2     3     1     4     7     0    -2    -4
4     5     6     2     5     8     2     0    -2
7     8     9     3     6     9     4     2     0
```

```
A6=cat(1, A1, A2, A3)
```

```
A6=
```

```
1     2     3
4     5     6
7     8     9
1     4     7
2     5     8
3     6     9
0    -2    -4
2     0    -2
4     2     0
```

一个多维数组 (包括矩阵) 的大小可以由 `size()` 函数来测出。

```
size(A) %显示矩阵的行数和列数
```

```
ans =
```

```
4     3
```

```
size(A4) %显示多维数组的各维大小
```

```
ans =
```

```
3     3     3
```

❖由矩阵编辑器创建和修改矩阵

当输入的矩阵很大，不适合用手工直接输入时，Matlab 提供了一个矩阵编辑器(Matrix Editor)来方便用户创建和修改比较大的矩阵。利用矩阵编辑器创建和修改某一矩阵。

由矩阵编辑器创建和修改矩阵步骤：

- (1)预先定义一个变量
- (2)单击 Workspace 使其处于焦点，选中该变量可以打开或删除该变量。
- (3)双击或点击 open selection 图标打开矩阵编辑器。
- (4)改变维数、元素的值。可以是数值也可以是表达式。

矩阵编辑器可以方便地创建和修改矩阵。还可以把原矩阵“裁减”为它的左上方的主子矩阵，或反过来把原矩阵“扩展”为更大的矩阵，Matlab 系统会自动地把扩展的部分设置为 0。

❖由函数创建和修改矩阵

Matlab 提供了大量的函数用于创建一些常用的特殊矩阵，下面分别介绍这些由函数创建和修改的矩阵。

(1). 常用特殊矩阵的生成

以下是 Matlab 中由于产生常用特殊矩阵的函数

zeros(m,n)	零矩阵
ones(m,n)	全部元素都为 1 的矩阵
eye(m,n)	单位阵
rand(m,n)	0—1 分布的随机矩阵
randn(m,n)	正态分布的随机矩阵
compan(A)	矩阵 A 的伴随矩阵
randperm(n)	产生 1~n 之间整数的随机排列
gallery	测试矩阵
hankel(m,n)	n 维 Hankel 矩阵
diag(A)	A 矩阵的对角阵
hilb(n)	n 维 Hilbert 矩阵
invhilb(n)	n 维逆 Hilbert 矩阵
magic(n)	n 维 Magic 魔方阵
toeplitz(m,n)	toeplitz 矩阵
wilkinson(n)	n 维 Wilkinson 特征值测试矩阵
handamard(n)	n 维 Handamard 矩阵
vander(A)	由矩阵 A 产生的 vandermonde 矩阵

例：1.2.7 产生一个 3×4 随机矩阵

```
R=rand(3,4)
```

R =

```
0.9501    0.4860    0.4565    0.4447
0.2311    0.8913    0.0185    0.6154
0.6068    0.7621    0.8214    0.7919
```

例：1.2.8 产生一个在区间[10, 20]内均匀分布的 4 阶随机矩阵

```

a=10;b=20;
x=a+(b-a)*rand(4)
x =
    19.2181    19.3547    10.5789    11.3889
    17.3821    19.1690    13.5287    12.0277
    11.7627    14.1027    18.1317    11.9872
    14.0571    18.9365    10.0986    16.0379

```

例：1.2.9 产生均值为 0.6，方差为 0.1 的 4 阶矩阵

```

mu=0.6; sigma=0.1;
x=mu+sqrt(sigma)*randn(4)
x =
    0.8311    0.7799    0.1335    1.0565
    0.7827    0.5192    0.5260    0.4890
    0.6127    0.4806    0.6375    0.7971
    0.8141    0.5064    0.6996    0.8527

```

例：1.2.10 0~1 分布的随机矩阵，用 rand 函数可以产生任意行列的 0~1 分布的随机矩阵。

```

ra=rand(2,4)
ra =
    0.2548    0.3659    0.1478    0.4586
    0.3654    0.5476    0.7843    0.2468

```

例：1.2.11 利用 diag 产生对角阵

对角阵是线性代数中常用的矩阵。Diag(M)是专门用来产生对角阵的函数，M 是一个矩阵或向量。对于矩阵 diag 函数取矩阵的对角元素产生一个列向量；而对于向量则产生一个对角阵。

```

a=randn(5,5);
d=diag(a);
D=diag(d);

```

结果如下：

```

a=
-0.2569  1.4784    -0.9862  0.2593    0.1474
-1.3649  1.5476  0.1572    1.2487   -1.3216
0.3214    -0.9875   -0.3548  0.0957    0.7214
0.8746    0.3369    3.1024   -0.5416    1.2485
-2.6547  0.9826    -0.1795   -0.2796
d=
-0.2569
1.5476
-0.5416
-0.6395
D=
-0.2569  0          0          0          0
0         1.5476    0          0          0
0         0         -0.3548  0          0
0         0         0         -0.5416  0

```

0 0 0 0 - 0.6395

(2). 矩阵标识和子矩阵

矩阵的子阵可以通过标量、向量、冒号的标识来引用和赋值

✧ 子阵的序号向量标识方式 $A(v,w)$

v,w 可以是任何排列的向量。 v,w 中任一个可以是冒号“:”，它表示全部行（在 v 的位置）或列（在 w 的位置）。

v, w 所用的值必须大于或等于 1 而小于或等于矩阵的维数，否则失败。

✧ “0~1” 向量标识方式 $A(l1,:)$ 、 $A(:,l2)$ 、 $A(l1,l2)$

向量 $L1, L2$ 的长度分别为 A 的行数和列数；向量 $L1, L2$ 中的元素或取 1（表示提取相应的行或列）或取 0（不提取）。

例 1.2.12 提取矩阵的子阵

(1) 产生一个五阶魔方阵 B ；

$B = \text{magic}(5)$ ；

(2) 提取 B 阵的第 1 行，第 2 行的第 1,3,5 个元素；

$B1 = B(1:2,[1,3,5])$ ；

(3) 提取 B 阵的第三行和第一行全部元素；

$B2 = B([3,1],:)$ ；

(4) 使得 B 阵的第一行和第三行第 2, 4 个元素为 0；

$B([1,3],[2,4]) = \text{zeros}(2)$ ；

(5) 标出 B 阵的第一行中小于 5 的元素；

$L = B(1,:) < 5$ ；

(6) 获得 B 阵的第一行中小于 5 的子向量；

$B3 = B(1,L)$ ；

(3). 矩阵的结构变换

可以通过矩阵的旋转、改变维数和截取部分元素来产生用户需要的新矩阵。Matlab 提供以下矩阵变换命令和函数：

$B = \text{rot90}(A)$ A 逆时针旋转 90° 得 B ；

$B = \text{rot90}(A,k)$ A 逆时针旋转 $k \times 90^\circ$ 得 B ；

$B = \text{fliplr}(A)$ B 由 A 左右翻转而得；

$B = \text{flipud}(A)$ B 由 A 上下翻转而得；

$B = \text{reshape}(A,m,n)$ B 阵的维数为 $(m \times n)$ ， $m \times n$ 等于 A 的列维和行维之积

$L = \text{reil}(A,k)$ L 第 k 条对角线以下的元素取 A 元素，其余为 0；

$L = \text{tril}(A)$ L 的主对角线元素取 A 元素其余为 0；

$U = \text{triu}(A)$ U 的主对角线元素取 A 元素其余为 0；

$U = \text{triu}(A,k)$ U 的第 k 条对角线以上的元素取 A 元素，其余为 0；

注： $k=0$ ，主对角线；

$k>0$ ，主对角线上面的第 k 条对角线；

$k<0$ ，主对角线下面的第 k 条对角线；

以下分别举例说明各函数的作用和用法。

例 1.2.13 矩阵的旋转和转置的区别

$A = [1,2,3,4;5,6,7,8;9,10,11,12]$

$B1 = \text{rot90}(A)$

$BT = A'$

$B2 = \text{rot90}(A,2)$

```

A=
    1     2     3     4
    5     6     7     8
    9    10    11    12

B1=
    4     8    12
    3     7    11
    2     6    10
    1     5     9

BT=
    1     5     9
    2     6    10
    3     7    11
    4     8    12

B2=
    12    11    10     9
     8     7     6     5
     4     3     2     1

```

例 1.2.14 矩阵的变维

B=reshape(A,2,6) % 矩阵 A 取自上例

```

B=
    1     9     6     3    11     8
    5     2    10     7     4    12

```

例 1.2.15 部分元素的截取

LA=tril(A,-1) %矩阵 A 取自上例

UA=triu(A,1)

```

LA=
    0     0     0     0
    5     0     0     0
    9    10     0     0

UA=
    0     2     3     4
    0     0     7     8
    0     0     0    12

```

(4). 矩阵的提取和保存

对于大型的项目来说，经常有大量的矩阵需要保存和再次使用时提取。当然最好的办法是把辛苦得来的数据保存于文件中。在 Matlab 中用 Mat 文件来保存二进制的数据。可以用 Mat 来保存和提取矩阵，具体步骤如下：

保存矩阵：如果 A,B 矩阵都已存在，用 save 命令保存，具体格式如下：

```
save mymatrix A B
```

说明：mymatrix 是用户自定义的文件名，Matlab 系统会自动地加上后缀.mat；而系统默认的路径为 matlabr11/bin，可以在前面加上路径而改变。

提取矩阵：在重新启动 Matlab 后，用 load 命令可以将保存在文件中的矩阵读到 Matlab 工作

区中来。

```
load mymatrix
```

load 命令不能指定变量名，系统仍将 A、B 作为矩阵的名称。

2. MATLAB 的数组的建立和保存

例 1.2.16 创建等差数列

方法一、 `a=0:0.5:10`

说明：以 “:” 间隔起始值(=0)、增量值(=0.5)、终止值(=10)；

如果增量值省略不写，默认增量值为 1；

方法二、 利用函数 `x=linspace(0,1,75)`

说明：利用函数 `linspace` 以间隔起始值(=0)、终止值(=1)和元素数目(=75)；

常用于绘图中区间的分割；

方法三、 从原来的数组创建新的数组

```
a=1:4; b=1:2:7;
```

```
c=[b,a]; %将两个数组 a,b 连接在一起。
```

```
c=1 3 5 7 1 2 3 4
```

```
d=[a(1:2:4),4 0.2 8];
```

```
d=1 3 4 .2 8
```

说明：数组的元素和矩阵一样可以通过下标来访问。

例 1.2.17 利用函数 `logspace` 创建等比数列

```
logspace(0,2,11)
```

```
ans =
```

```
Columns 1 through 7
```

```
1.0000 1.5849 2.5119 3.9811 6.3096 10.0000 15.8489
```

```
Columns 8 through 11
```

```
25.1189 39.8107 63.0957 100.0000
```

说明：该函数产生一个起点为 100、终点为 102、包含 11 个数据的等比数列。

3. MATLAB 的矩阵运算和数组运算

(1) 矩阵运算：

例 1.2.18 矩阵相加运算

```
A=[1, 1, 1; 1, 2, 3; 1, 3, 6]
```

```
B=[8, 1, 6; 3, 5, 7; 4, 9, 2]
```

```
C=A+B
```

```
D=A-B
```

结果显示：C=

```
9 2 7
4 7 10
5 12 8
```

```

D=
    -7     0    -5
    -2    -3    -4
    -3    -6     4

```

例 1.2.19 矩阵乘法运算

```

X= [2  3  4  5;
    1  2  2  1];
Y=[0  1  1;
   1  1  0;
   0  0  1;
   1  0  0];
Z=X*Y
a=2*X           %矩阵的数乘
结果显示为：

```

```

Z=
     8     5     6
     3     3     3

a =
     4     6     8    10
     2     4     4     2

```

例 1.2.20 矩阵的点积运算

```

X=[-1  0  2];
Y=[-2  -1  1];
Z=dot(X, Y)

```

则显示：Z =

```

4

```

还可用另一种算法：

```

sum(X.*Y)
ans=
4

```

例 1.2.21 向量的叉乘：计算垂直于向量(1, 2, 3)和(4, 5, 6)的向量。

```

a=[1  2  3];
b=[4  5  6];
c=cross(a,b)

```

结果显示：

```

c=
    -3     6    -3

```

可得垂直于向量(1, 2, 3)和(4, 5, 6)的向量为±(-3, 6, -3)

例 1.2.22 混合积：计算向量 a=(1, 2, 3)、b=(4, 5, 6)和 c=(-3, 6, -3) 的混合积 a·(b×c)

```

a=[1  2  3]; b=[4  5  6]; c=[-3  6  -3];
x=dot(a, cross(b, c))

```

结果显示：x =

```

54

```

注意：先叉乘后点乘，顺序不可颠倒。

例 1.2.23 展开多项式 $(s^2 + 2s + 2)(s + 4)(s + 1)$

```
w=conv([1,2,2],conv([1,4],[1,1]))
w =
     1     7    16    18     8
P=poly2str(w,'s')    %将 w 表示成多项式
P =
s^4 + 7 s^3 + 16 s^2 + 18 s + 8
```

例 1.2.24 $A=[3,6,7;9,2,5;1,6,3]$ ，矩阵的非整数乘方 $Ap1=A^{0.3}$

```
Ap1=A^0.3
Ap1=
    1.4908-0.0000i    0.2551-0.0000i    0.6711+0.0000i
    2.1879+0.0000i    0.5145-0.0000i   -0.6590+0.0000i
   -1.4515-0.0000i    1.3756-0.0000i    2.2378+0.0000i
```

标量的矩阵乘方 $pA1=(0.5)^A$

```
pA1=(0.5)^A
pA1=
    1.4876+0.0000i    1.2124-0.0000i   -2.5695-0.0000i
   -13.2981-0.0000i    8.4944+0.0000i    6.7506+0.0000i
    11.3044+0.0000i   -8.2394+0.0000i   -4.1478-0.0000i
```

数组的标量乘方 $Ap2=A.^{0.4}$

```
Ap2=A.^0.4
Ap2=
    1.5518    2.0477    2.1779
    2.4082    1.3195    1.9037
    1.0000    2.0477    1.5518
```

标量的数组乘方 $pA2=0.4.^A$

```
pA2=0.4.^A
pA2=
    0.0640    0.0041    0.0016
    0.0003    0.1600    0.0102
    0.4000    0.0041    0.0640
```

说明：无论那种运算都是对元素逐个进行的。

(2) 数组运算：

建立两个矩阵并对其进行如下运算：.+加法，.-减法，.*乘法，.^幂，.\左除，./右除，数组乘方，数组的方幂。

(3) 矩阵运算的应用——线性方程组的求解

例 1.2.25

$$\text{求方程组} \begin{cases} 5x_1 + 6x_2 & = 1 \\ x_1 + 5x_2 + 6x_3 & = 0 \\ x_2 + 5x_3 + 6x_4 & = 0 \\ x_3 + 5x_4 + 6x_5 & = 0 \\ x_4 + 5x_5 & = 1 \end{cases} \text{的解。}$$

解：

```
A=[5 6 0 0 0
    1 5 6 0 0
    0 1 5 6 0
    0 0 1 5 6
    0 0 0 1 5];
B=[1 0 0 0 1]';
R_A=rank(A)      %求秩
X=A\B            %求解
```

运行后结果如下

```
R_A =
     5
X =
    2.2662
   -1.7218
    1.0571
   -0.5940
    0.3188
```

这就是方程组的解。

或用函数 rref 求解：

```
C=[A,B]          %由系数矩阵和常数列构成增广矩阵 C
R=rref(C)         %将 C 化成行最简行
R =
    1.0000         0         0         0         0    2.2662
         0    1.0000         0         0         0   -1.7218
         0         0    1.0000         0         0    1.0571
         0         0         0    1.0000         0   -0.5940
         0         0         0         0    1.0000    0.3188
```

则 R 的最后一列元素就是所求之解。

例 1.2.26

$$\text{求方程组} \begin{cases} x_1 + x_2 - 3x_3 - x_4 = 1 \\ 3x_1 - x_2 - 3x_3 + 4x_4 = 4 \\ x_1 + 5x_2 - 9x_3 - 8x_4 = 0 \end{cases} \text{的一个特解。}$$

解：

```
A=[1 1 -3 -1;3 -1 -3 4;1 5 -9 -8];
B=[1 4 0]';
X =A\B            %由于系数矩阵不满秩，该解法可能存在误差。
    = [ 0  0 -0.5333  0.6000]' (一个特解近似值)。
```

若用 rref 求解，则比较精确：

```

A=[1 1 -3 -1;3 -1 -3 4;1 5 -9 -8];
B=[1 4 0]';
C=[A,B];           %构成增广矩阵
R=rref(C)
R =
    1.0000         0   -1.5000    0.7500    1.2500
         0    1.0000   -1.5000   -1.7500   -0.2500
         0         0         0         0         0

```

由此得解向量 $X=[1.2500 \quad -0.2500 \quad 0 \quad 0]'$ (一个特解)。

三. MATLAB 程序设计

例 1.3.1 字符变量的输入和检查

```

a= ' this is a string '
isstr(a)
ans=1

```

例 1.3.2 考虑建立学生档案结构体，这里面应包含下列信息：

编号(用 number 表示，而在结构体下 number 又称为其成员变量或域)：

- 姓名(name)：字符串型。
- 身高(height)：数值型。
- 考试成绩(test)：矩阵，其第 i 行为第 i 次考试的成绩，而第 j 列为第 j 门考试的成绩。

有了上述要求，就可以由下面的语句建立起结构体 student_rec：

```

student_rec.number=1;
student_rec.name= ' 张三 ' ;
student_rec.height=180;
student_rec.test=[100, 80, 75; 77,60,92;
                  67,28,90; 100,89,78];

```

student_rec %显示结构体的内容

```

student_rec=
    number:1
    name: ' 张三 '
    height:180
    test:[4x3 double]

```

和 C 语言不同，引用结构体中成员变量时直接引用即可，而无需用->算符来引用。例如若想获得该结构体中的 test 成员变量，则可以通过下面的语句来直接得出。

```

student_rec.test
ans=
    100     80     75
    77     60     92
    67     28     90
    100     89     78

```

下面的命令建立一组 100 个学生(假设 2 个年级，每个年级 50 人)的新的结构体变量 b：

```

b(50, 2)=struct(student_rec) %构造 50x2 结构体矩阵

```

```

b=
    50x2 struct array with fields:
        number
        name
        height
        test

```

这样就可以构成一个小型的数据库了。数据库中每个条目的具体内容还需要根据实际情况填写，填写的格式是很简单的，例如第 2 年级第 43 个学生的有关信息可以由下面的语句填写。

```

b(43, 2). Number=50+43;
b(43, 2). Name= ' 李四 ';
b(43, 2). Height=186;
b(43, 2). Test=[83, 80, 78; 97, 80, 72;
                69, 88, 80; 87, 99, 100];

```

结构体的结构也是可以修改的，例如若想给 b 变量加一个成员变量——体重(weight)，则可以将结构体中任意一个变量后面加一个 weight 成员变量，即

```

b(1, 1). weight=90 %添加成员变量 weight
b=
    50x2 struct array with fields:
        number
        name
        height
        test
        weight

```

这样就可以把所有的 b 结构体变量均加一个 weight 成员变量，但每个其他成员函数均为空矩阵，可以使用赋值语句给它赋予确定的值。

如果想消除结构体变量中某一成员变量，则可以使用 rmfield() 函数来完成。例如，如果想消除 b 结构体中的 weight 成员变量，则可以给出如下语句：

```

b=rmfield(b, ' weight ') %删除成员变量 weight
b=
    50x2 struct array with fields:
        number
        name
        height
        test

```

例 1.3.3 这里重新考虑例 3.2 中提出的问题，可以用单元数据结构来构造某个学生的档案。

```

B={1, ' 张三 ', 180, ...
   [100, 80, 75; 77, 60, 92; 67, 28, 90; 100, 89, 78]}
B=
    [1]      ' 张三 '      [180]      [4x3double]
size[B]
ans =
    1     4

```

其今…为续行符号，表示下一行的内容紧跟上行，这样就可以将长的 MATLAB 语句分成几行

来表示。可见，对单元变量也可以使用 `size()` 和 `length()` 函数。从上面的显示中，并不能直接看出此单元数据的第 4 个元素代表的矩阵具体内容是什么，只能定性地看出它是一个 4x3 矩阵。

可以由带有大括号下标的形式访问该单元数据，得出该矩阵的具体内容为：

```
B{4}           %显示第 4 单元内容
```

```
ans=
    100     80     75
     77     60     92
     67     28     90
     100     89     78
```

注意，不能用圆括号去访问单元数组。另外，上面得出的 `ans` 结果不再是单元数组，而是常规的矩阵了。也可以使用 `celldisp()` 函数来直接显示整个单元。

```
Celldisp(B) %显示整个单元变量
```

```
B{1}=
     1
B{2}=
    张三
B{3}=
    180
B{4}=
    100     80     75
     77     60     92
     67     28     90
     100     89     78
```

可以消除单元变量中的某个单元元素。例如，如果想删除 B 单元变量的第 3 个单元元素，则可以给出下面的命令：

```
B(3) = []
B =
     [1]          '张三'      [4x3 double]
```

可见，通过这个命令就消除了原单元数据 B 的第 3 个单元。注意，这里用的是 `B(3)` 而不是 `B{3}`，后者只能将第 3 个单元置成空矩阵，而不能消除它，所以这里的表示方法是 MATLAB 的约定，不一定有其他合适的解释。

例 1.3.4 计算向量元素的平均值

(1) 在字处理软件中编写以下内容：

```
function y=average(x)
% 向量元素的平均值
% 语法：average(x)，其中 x 为输入向量
% 当输入非向量时，给出错误信息
[m,n]=size(x)
if ~(m==1)|(n==1)|(m==1&n==1)
error(' Input must be a vector ') % 判断输入是否为向量
end
y=sum(x)/length(x);           % 实际计算过程
```

(2) 将 `average.m` 存盘。该文件定义了名为 `average` 的新函数。此函数的用法与别的函数一样。

(3) 在指令窗中运行以下命令，可以求得 1~100 的平均值。

```
average(1:100)
ans =
    50.5000
```

(4) 联机查阅 average 函数的使用说明。在指令窗口中运行 help 指令，得到帮助信息。

```
help average
% 向量元素的平均值
% 语法：average(x)，其中 x 为输入向量
% 当输入非向量时，给出错误信息
```

例 1.3.5 计算 Matlab 中特殊值 eps。

% Example2.3.3.1 计算 Matlab 中特殊 eps 值。

```
EPS=1;
num=0;
while (1+EPS>1)
    EPS = EPS/2;
    num=num+1;
end
num
EPS=2*EPS
```

执行后得

```
num =
    53

EPS =
    2.2204e-016
```

例 1.3.6 求 EPS 的另一种方法

% Example2.3.3.2 计算 EPS 的另一种方法。

```
EPS=1;
for num=1:100
    EPS = EPS/2;
    if (1+EPS)<=1
        EPS = 2*EPS;
        break;
    end;
end
```

结果与上例相同。

注意：break 出现在嵌套的 for 或 while 循环结构里，那么 matlab 只跳出 break 所在的那个循环里，而不跳出整个嵌套结构。

例 1.3.7 折扣问题。购买 2.5 元/斤的苹果，如果购买量超过 50 斤，给 20%的折扣；超过 100 斤，给 30%的折扣。

% Example2.3.3.3 折扣问题。

```
apples = 100;          % apple 数目
Cost = apples*2.5;      % 购买 apple 的费用
Cost = 250
If(apples>50)          % 购买量超过 50 斤，给 20%折扣
```

```

cost = (1-20/100)*cost ;
elseif (apple>100)          % 超过 100 斤，给 30%折扣。
    cost =(1-30%)*cost
end

```

例 1.3.8 用 eval 产生 5 阶的 Hilbert 矩阵。

```

n=5
t='1/(l+j-1)';
a=zeros(n);
for i=1:n
    for j=1:n
        a(i,j)=eval(t);
    end
end
a

```

Hilbert 矩阵

$$\begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} \\ \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} \\ \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} & \frac{1}{9} \end{bmatrix}$$

运行结果为：

```

a=
    1.0000    0.5000    0.3333    0.2500    0.2000
    0.5000    0.3333    0.2500    0.2000    0.1667
    0.3333    0.2500    0.2000    0.1667    0.1429
    0.2500    0.2000    0.1667    0.1429    0.1250
    0.2000    0.1667    0.1429    0.1250    0.1111

```

例 1.3.9 feval 使用

% Example 2.3.5.2 feval 函数的使用

```
disp(' 函数对应索引值为 sin.....1, cos.....2, log.....3, ')
```

```
fun=['sin';'cos';'log'];
```

```
k=input('选择函数号：')
```

```
x=input('')
```

```
feval(fun(k,:),x);
```

运行后指令窗口显示：

函数对应索引值为 sin.....1, cos.....2, log.....3,

选择函数号：

输入 3 后，指令窗口显示：

输入的值：

输入 2 后得出结果：

```
ans=  
    0.3010
```

例 1.3.10 用递归调用形式计算 n 的阶乘

(1)编写函数 factor.m 编写递归调用函数 factor.m

```
function f=factor(n)  
% factor.m    计算 n 的阶乘  
if n==1  
    f=1;  
    return;  
else  
    f=n*factor(n-1)  
    return;  
end
```

(2)运行 factor(6) 运行函数文件

```
factor(6)  
ans=  
    720
```

例 1.3.11 编写函数 M 文件并对其进行调试。

(1) 函数 M 文件的编写

新建 2 个 M 文件 test.m 和 test1.m，准备开始调试。

test.m 的内容为：

```
function a=test(b)  
c=sqrt(b)*cos(b)  
a=test1(b,c)
```

test1.m 的内容为

```
function a=test1(b,c)  
q=cond(b)  
[w,e]=eig(c)  
a=w*q
```

(2) 设置断点

确认将文件放在搜索路径下，在 Matlab 主窗口中输入：

```
edit cond  
edit test  
edit test1
```

在 test.m 的第 2 行($c=\sqrt{b} \cdot \cos(b)$)、cond.m 的第 36 行($s=\text{svd}(A);$)设置断点。

(3) 启动 m 文件及察看堆栈

在 Matlab 的主窗口中输入：`test(magic(4))`

可以看到程序停在 test.m 的第 2 行，并且工具栏最右边的列表栏中显示目前所在的工具区为 test/Base Workspace。说明这时还没有进行其它的调用过程。对于这一点，可以在 Matlab 主窗口中输入 dbstack 指令来验证。

选择 Debug 菜单的 Continue 命令，继续运行至 cond.m 的第 36 行停下。这时可以看到，工作区按 cond/test1/Base Workspace 来排列，而这正是调用的先后顺序。

回到 Matlab 主窗口，输入 who，可知当前变量为

Your variables are :

A m n p

输入 A，可以看到 A 的内容。这里，A 是一个由 magic(4) 产生的魔方阵。

A =

```
16     2     3    13
 5    11    10     8
 9     7     6    12
 4    14    15     1
```

选择 Debug 菜单的 Single Step 命令，执行第 36 行，并且停在第 37 行。

```
if any(s == 0) % Handle singular matrix
```

检查变量 s，为

s =

```
34.0000    17.8885     4.4721     0.0000
```

这时，再次查看 cond 的工作区，发现新增加了变量 s。

whos

Name	Size	Bytes	Class
A	4x4	128	double array
m	1x1	8	double array
n	1x1	8	double array
p	1x1	8	double array
s	4x1	32	double array

(4) 改变工作区

从列表栏中选择相应的名称，就可以改变函数所处的工作区，相当于 dbup 和 dbdown 的组合。对应于每一个工作区，都可以查看、修改变量内容。

现在，从列表栏中选择 Base Wordkpace，将工作区转到 Matlab 的工作区，即调用函数 test.m 的工作区。

逐步执行函数，直至 cond.m 函数结束，转到 test1.m。在此过程中，可以看到工作区和变量的作用域的改变。

如果没有发现错误，可以退出调试状态。

实验二．控制系统时域分析法

例 2.1 典型二阶系统

$$H(s) = \frac{\omega_n^2}{s^2 + 2\xi\omega_n s + \omega_n^2}$$

其中 ω_n 为自然频率（无阻尼振荡频率）， ξ 为相对阻尼系数。试绘制出当 $\omega_n=6$ ， ξ 分别为 0.1, 0.2, ……，1.0, 2.0 时的单位阶跃响应。

解 MATLAB 程序为 ex2.1.m：

```
%example2.1
Wn = 0.6
kesai = [0.1:0.1:1,2]
hold on;
for kos=kesai
    num=Wn^2;
    den=[1,2*kos*Wn,Wn^2]
    step(num,den)
end
title('Step Response')
hold off;
```

执行后得单位阶跃响应曲线图，从图中可以看出，在过阻尼和临界阻尼响应曲线中，临界阻尼响应具有最短的上升时间，响应速度最快；在欠阻尼（ $0 < \xi < 1$ ）响应曲线中，阻尼系数越小，超调量越大，上升时间越短，通常取 $\xi=0.4-0.8$ 为宜，这时超调量适度，调节时间较短。

例 2.2 对例 2.1 中的典型二阶系统，绘制出当 $\xi=0.7$ ， ω_n 取 2, 4, 6, 8, 10, 12 时的单位阶跃响应。

解 MATLAB 程序为 ex2.2.m：

```
%example2.2
w=[2:2:12]
kesai=0.7
hold on;
for Wn=w
    num=Wn^2;
    den=[1,2*kesai*Wn,Wn^2]
    step(num,den,6)
end
title('Step Response')
hold off;
```

执行后得单位阶跃响应曲线图，从图中可以看出， Wn 越大，响应速度越快。

例 2.3 求三阶系统

$$H(s) = \frac{5(s^2 + 5s + 6)}{s^3 + 6s^2 + 10s + 8}$$

的单位阶跃响应。

解 MATLAB 程序为 ex2.3.m :

```
%example3.10
num=[5 25 30]
den=[1 6 10 8]
step(num,den)
title('Step Response')
```

例 2.4 求典型二阶系统

$$H(s) = \frac{\omega_n^2}{s^2 + 2\xi\omega_n s + \omega_n^2}$$

当 $\xi=0.7$, $\omega_n=6$ 时的单位冲激响应。

解 MATLAB 程序为 ex2.4.m :

```
Wn = 6
kesai = 0.7
hold on;
num=Wn^2;
den=[1,2*kesai*Wn,Wn^2]
step(num,den)
title('Step Response')
```

例 2.5 有高阶系统

$$\begin{cases} \dot{x} = \begin{bmatrix} -1.6 & -0.9 & 0 & 0 \\ 0.9 & 0 & 0 & 0 \\ 0.4 & 0.5 & -5.0 & -2.45 \\ 0 & 0 & 2.45 & 0 \end{bmatrix} x + \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} u \\ y = [1 \ 1 \ 1 \ 1]x \end{cases}$$

求单位阶跃响应, 单位冲激响应及零输入响应 (设初始状态 $x_0=[1 \ 1 \ 1 \ -1]^T$)。

解: MATLAB 程序为 ex2.5.m :

```
%example 2.5
a0=[-1.6,-0.9,0,0;0.9,0,0,0; 0.4,0.5,-5,-2.45;0,0,2.45,0]
b0=[1;0;1;0]
c0=[1,1,1,1]
d0=[0]
t=0.5
[a,b,c,d]=c2dm(a0,b0,c0,d0,t,'tustin');
figure(3)
subplot(2,2,1)
dstep(a,b,c,d)
```

```

title('Descrete Step Responce')
subplot(2,2,2)
dimpulse(a,b,c,d)
title('Descrete Impluse Responce')
subplot(2,2,3)
x0=[1;1;1;-1]
dinitial(a,b,c,d,x0)
axis([0 6 -0.5 2.5])
title('Descrete Initial Responce')
subplot(2,2,4)
[z,p,k]=ss2zp(a,b,c,d,1)
zplane(z,p)
title('Descrete Ploe-Zero Map')

```

例 2.6 多输入多输出系统

$$\begin{cases} \dot{x} = \begin{bmatrix} 2.25 & -5 & -1.25 & -0.5 \\ 2.25 & -4.25 & -1.25 & -0.25 \\ 0.25 & -0.5 & -1.25 & -1 \\ 1.25 & -1.75 & -0.25 & -0.75 \end{bmatrix} x + \begin{bmatrix} 4 & 6 \\ 2 & 4 \\ 2 & 2 \\ 0 & 2 \end{bmatrix} u \\ y = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 2 & 0 & 2 \end{bmatrix} x \end{cases}$$

求单位阶跃响应和单位冲激响应。

解:这是 2 输入 2 输出系统，因此，其阶跃响应和冲激响应各有 4 个。MATLAB 中的 step 和 impulse 函数本身可处理多输入多输出情况，因此，编写 MATLAB 的程序是很简单的。

MATLAB 程序为 ex2.6.m：

```

% example2.6
a=[2.25 -5 -1.25 -0.5;2.25 -4.25 -1.25 -0.25;0.25 -0.5 -1.25 -1;1.25 -1.75 -0.25 -0.75]
b=[4 6;2 4;2 2;0 2]
c=[0 0 0 1;0 2 0 2]
d=zeros(2,2)
figure(1)
step(a,b,c,d)
figure(2)
impulse(a,b,c,d)

```

例 2.7 将例 2.5 中的连续系统，以 $t=0.5$ 取样周期，采用双线性变换算法转换成离散系统，然后求出离散系统的单位阶跃响应、单位冲激响应及零输入响应（设初始状态 $x_0=[1 \ 1 \ 1 \ -1]^T$ ）。

解 MATLAB 程序为 ex2.7.m：

```

%example2.7
a=[-1.6 -0.9 0 0;0.9 0 0 0;0.4 0.5 -5 -2.45;0 0 2.45 0]
b=[1;0;1;0]
c=[1 1 1 1]
d=[0]

```

```

t=0.5
[a,b,c,d]=c2d(a1,b1,c1,d1,t,t,'tustin')
figure(1)
subplot(2,2,1)
dstep(a,b,c,d)
title(Discrete Step Response)
subplot(2,2,2)
dimpulse(a,b,c,d)
title(Discrete Impulse Response)
subplot(2,2,3)
x0=[1;1;1;-1]
dinitial(a,b,c,d,x0)
axis([0 6 -0.5 2.5])
title(Discrete Initial Response)
subplot(2,2,4)
[z,p,k]=ss2zp(a,b,c,d,1)
title(Discrete Pole -Zero Map)

```

实验三．控制系统的根轨迹法

例 3.1 设开环系统

$$H(s) = \frac{k(3s+1)}{s(2s+1)}$$

绘制出通过单位负反馈构成的闭环系统的根轨迹。

解 可直接利用 rlocus 函数绘制出根轨迹。MATLAB 程序为 ex3.1.m：

```
num=[3,1]
den=[2,1,0]
rlocus(num,den)
sgrid
title('根轨迹')
```

例 3.2 设开环系统

$$H(s) = \frac{k(s+5)}{s(s+2)(s+3)}$$

绘制出闭环系统的根轨迹，并确定交点处的增益 k_0 。

解：利用 rlocus 函数绘制出根轨迹，利用 rlocfind 函数可找出根轨迹上任一点处的增益。因此 MATLAB 程序为 ex3.2.m：

```
num=[1,5]
den=[1,5,6,0]
rlocus(num,den)
sgrid
title('根轨迹')
[k,p]=rlocfind(num,den)
gtext('k=0.5')
```

这说明系统有三个极点。实际上，如果能将十字光标准确放在交点时，则 $p(2) = p(3)$ 。最后得到了如图 3.12 所示的根轨迹。

例 3.3 已知开环传递函数为

$$H(s) = \frac{k}{s^4 + 16s^3 + 36s^2 + 80s}$$

绘制出闭环系统的根轨迹。

解 MATLAB 程序为 ex3.3.m：

```
num=[1]
den=[1,16,36,80,0]
rlocus(num,den)
title('根轨迹')
```

例 3.4 已知开环传递函数为

$$H(s) = \frac{k(s+2)}{(s^2 + 4s + 3)^2}$$

绘制出闭环系统的根轨迹，并分析其稳定性。

解:MATLAB 程序为 ex3.4.m：

```
%example3.4
num=[1,2]
den=[1,4,3]
figure(1);
rlocus(num,den)
title('根轨迹')
[k,p]=rlocfind(num,den)
%check the stability
figure(2);
k=55;
num1=k*[1 2]
den=[1 4 3]
den1=conv(den,den);
[num,den]=cloop(num1,den1,-1);
impulse(num,den);
title('Impulse Response(k=55)')
%check the stability
figure(3);
k=56;
num1=k*[1 2]
den=[1 4 3]
den1=conv(den,den);
[num,den]=cloop(num1,den1,-1);
impulse(num,den);
title('Impulse Response(k=56)')
```

实验四．控制系统反馈校正设计

例 4.1 被控对象 $G(s) = \frac{10}{(s+1)(s+2)(s+3)}$ ，设计反馈控制器 $u=-kx$ ，使闭环系统的极

点为 $\mu_1 = -2 + j2\sqrt{3}$ ， $\mu_2 = -2 - j2\sqrt{3}$ ， $\mu_3 = -10$ 。

解：定义状态变量

$$x_1 = y$$

$$x_2 = \dot{x}_1$$

$$x_3 = \dot{x}_2$$

因此，系统的状态方程为

$$\begin{cases} \dot{x} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -6 & -11 & -6 \end{bmatrix} x + \begin{bmatrix} 0 \\ 0 \\ 10 \end{bmatrix} u \\ y = [1 \ 0 \ 0] \end{cases}$$

然后根据变换法和 Ackermann 公式求出所设计的增益阵 k ，这里先给出按这两种方法求解的 Matlab 程序，然后给出工具箱函数 place（或 acker）求解的结果，它们是一致的。

Matlab 程序为 ex4.1.m

```
%
% example4.1
% Pole Placement ..... using transformation matrix
%
disp(' Pole Placement ..... using transformation matrix ')
a=[0 1 0;0 0 1;-6 -11 -6];
b=[0;0;10]
% Step1
cam=ctrb(a,b);
disp(' The rank of controllability matrix ');
rc=rank(cam)
% Step2
beta=poly(a)
% Step3
a1=beta(2);a2=beta(3);a3=beta(4);
w=[a2 a1 1;a1 1 0;1 0 0];
t=cam*w;
% Step4
j=[-2+2*sqrt(3)*i 0 0;
    0 -2-2*sqrt(3)*i 0;
    0 0 -10];
alph=poly(j);
```

```

aa1=alph(2);aa2=alph(3);aa3=alph(4);
% Step5
k=[aa3-a3 aa2-a2 aa1-a1]*(inv(t))
%
% Pole Placement ..... using Ackermann's formula
%
disp(' Pole Placement ..... using Ackermann's formula ');
a=[0 1 0;0 0 1;-6 -11 -6];
b=[0;0;10];
% Step1
cam=ctrb(a,b);
disp(' The rank of controllability matrix ');
rc=rank(cam)
% Step2
j=[-2+2*sqrt(3)*i 0 0
0 -2-2*sqrt(3)*i 0;
0 0 -10];
alph=poly(j);
% Step3
phi=polyvalm(alph,a);
% Step4
k=[0 0 1]*(inv(cam))*phi
%
% Pole Placement ..... using place function in Matlab
%
p=[-2+2*sqrt(3)*i -2-2*sqrt(3)*i -10];
k=place(a,b,p)

```

执行后得

Pole Placement using transformation matrix
The rank of controllability matrix

rc=

3

beta=

1.0000 6.0000 11.0000 6.0000

alph=

1.0000 14.0000 56.0000 160.0000

k=

15.4000 4.5000 0.8000

Pole Placement using Ackermann's formula

The rank of controllability matrix

rc=

3

```
k=
15.4000 4.5000 0.8000
```

Pole Placement using place function in Matlab
Place:ndigits =17

```
k=
15.4000 4.5000 0.8000
```

例 4.2 开环系统

$$\begin{cases} \dot{x} = ax + bu \\ y = cx \end{cases}$$

其中

$$a = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -6 & -11 & -6 \end{bmatrix} \quad b = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad c = [1 \quad 0 \quad 0]$$

设计全阶状态观测器，使观测器的闭环极点为 $-2 \pm j2\sqrt{3}$ ，5

解：状态观测器的机构见图 4.2。为求出状态观测器的反馈增益阵 k_e ，先为原系统改造一对偶系统

$$\begin{aligned} \dot{p} &= a' p + c' v \\ q &= b' p \end{aligned}$$

然后采用极点配置的方法对对偶系统进行闭环极点位置的配置，得到反馈增益阵 k ，从而可由对偶原理得到原系统的状态观测器增益阵 $k_e = k'$ 。

Matlab 程序为 ex4.2.m

```
% Example 4.2
%
% Design of a full.....order state observer
%
a=[0 1 0;0 0 1;-6 -11 -6];
b=[0;0;1]
c=[1 0 0]
% Check obserbability
disp('The rank of obserbability matrix')
ro=rank(observ(a,c))
% Construct the dual system (a1,b1,c1)
a1=a';
b1=c';
c1=b';
% Solve K using poles placement.....Ackermann
p=[-2+2*sqrt(3)*i -2-2*sqrt(3)*i -5];
k=acker(a1,b1,p1)
% Obtain the state observer gain Ke
```

ke=k'

执行后得

The rank of observability matrix

ro=

3

ke=

3.0000

7.0000

-1.0000

由于 ro=3, 所以系统可观, 由此可设计全维状态观测器。

例 4.3 最小阶状态观测器设计。设开环系统

$$\begin{cases} \dot{x} = ax + bu \\ y = cx \end{cases}$$

其中

$$a = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -6 & -11 & -6 \end{bmatrix} \quad b = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad c = [1 \quad 0 \quad 0]$$

假定输出 y (即 x_1) 可测, 设计最小阶状态观测器, 使闭环极点为 $-2 \pm j2\sqrt{3}$ 。

解: 由于 x_1 可测, 由此只需设计 x_2 和 x_3 的状态观测器, 故原系统可分成两部分:

可测部分: $\dot{x}_a = a_{aa}x_a + a_{ab}x_b + b_a u$

不可测部分: $\dot{x}_b = a_{ba}x_a + a_{bb}x_b + b_b u$

其中

$$\begin{aligned} a_{aa} &= 0 & a_{ab} &= [1 \quad 0] \\ a_{ba} &= \begin{bmatrix} 0 \\ -6 \end{bmatrix} & a_{bb} &= \begin{bmatrix} 0 & 1 \\ -11 & -6 \end{bmatrix} \\ b_a &= 0 & b_b &= \begin{bmatrix} 0 \\ 1 \end{bmatrix} \end{aligned}$$

根据上述结论, 等效系统为

$$\begin{cases} \dot{p} = a_c p + b_c v \\ q = c_c p \end{cases}$$

其中 $a_c = a_{bb}$ $b_c v = a_{ba}x_a + b_b u$ $c_c = a_{ab}$

由此可方便的利用 Matlab 的 acker 或 place 函数进行设计

Matlab 程序 ex43

% Example 4.3

%

```

% Design of a minimum-order observer
%
a=[0 1 0;0 0 1;-6 -11 -6];
b=[0;0;1];c=[1 0 0]
aaa=[a(1,1)];aab=[a(1,2:3)];
aba=[a(2:3,1)];abb=[a(2:3,2:3)];
ba=b(1,1);bb=b(2:3,1);
% subsystem
a1=abb;c1=aab;
disp('Design of a minimum-order observer');
ro=rank(observ(a1,c1))

% Construct dual system
ax=a1 ' ;
bx=c1 ' ;
p=[-2+2*sqrt(3)*i -2-2*sqrt(3)*i];
k=acker(ax,bx,p);
% State observer gain
ke=k '

```

执行后得

The rank of observability matrix

ro=

2

ke=

-2

17

实验五．对控制系统利用 SIMULINK 进行仿真

1. 选择某一控制系统进行分析，可以选择三容系统或小车倒立摆系统或其它控制系统（教师可以根据实际情况任选），对控制系统建模并设计控制器。；
2. 在 SIMULINK 环境中进行系统的仿真以及性能分析或校正。
 要求： 可以利用 SIMULINK 进行系统仿真、设计以及调试。